Ministry of Higher Education and Scientific Research

University of Mannouba

National School of Computer Sciences



Design & Development Project

Report

Mobile Shopping Assistant

Elaborated by

Rihab Ben Rhouma

Meriem Harzallah

Supervised by

Mrs Leila Jemni Ben Ayed

Year 2011-2012


Supervisor's signature

ACKNOWLEDGEMENTS

We would like to give special thanks to everyone who has helped us make our idea come to life, we are extremely grateful that we got a chance to gain more experience in the field of software development by putting our knowledge into practice.

This project has brought us closer to a lot of people, especially with the help of our amazing supervisor Mrs Leila Jemni Ayed who has stood by us through every step process and kept supporting us all the way.

We would also like to thank our school administration for being so kind as to let us work on this subject in particular and let us exploit our potential to the fullest.

TABLE OF CONTENTS

GENERAL INTRODUCTION ____ *10*

CHAPTER 1 PROJECT PRESENTATION ____ *11*

1.1 General Concepts ____ **11**

1.1.1 Mobility ____ 11
1.1.2 Location-awareness ____ 11
1.1.3 Location-based Services ____ 12
1.1.4 Mobile Application Definition ____ 13
1.1.5 Mobile Application Development ____ 13
1.1.6 Mobile Marketing ____ 13
1.1.7 Web Services ____ 14

1.2 State of the Art ____ **15**

1.3 Conclusion ____ **16**

CHAPTER 2 REQUIREMENTS SPECIFICATION ____ *17*

2.1 Functional Requirements: ____ **17**

2.1.1 Identifying the Actors ____ 17

2.2 Nonfunctional Requirements Specifications: ____ **20**

2.3 Use case analysis ____ **20**

2.4 Principal Scenarios ____ **22**

2.4.1 Database Administrator ____ 22
2.4.2 Supermarket Manager ____ 23
2.4.3 Customer ____ 24

2.5 Conclusion ____ **24**

CHAPTER 3 DESIGN ____ *25*

3.1 General Design ____ **25**

3.1.1 Potential Architectures: ____ 25
3.1.2 Potential Architecture Patterns ____ 28

3.2 Detailed Design ____ **31**

3.2.1 Database administration ____ 31
3.2.2 Mobile Client Navigation Diagram ____ 35
3.2.3 Web Client Navigation Diagram ____ 35

3.2.4 Class Diagram for the Web Application 37
3.2.5 Class Diagram for the Mobile Application 38
3.2.6 Detailed scenarios 39

3.3 Conclusion 42

CHAPTER 4 IMPLEMENTATION 43

4.1 Workspace Environment: 43
4.1.1 Hardware Environment 43

4.2 Software Environments and Technological Choices 43
4.2.1 The Client-Side (Front-End): Mobile Development 44
4.2.2 The Server-Side (Back-End): Web Development 47
4.2.3 Web Services deployment: 49
4.2.4 Other Softwares used 51

4.3 Final results presentation 51
4.3.1 Android client perspective 51
4.3.2 Web client perspective 59

4.4 Project Timeline 68

4.5 Problems faced during the project development 68

4.6 Possible Future Improvements 69

4.7 Conclusion 70

GENERAL CONCLUSION 71

BIBLIOGRAPHY 73

NETOGRAPHY 74

ANNEX A INDOOR POSITIONING TECHNIQUES 75

ANNEX B INDOOR POSITIONING API 79

ANNEX C REPRESENTATIONAL STATE TRANSFER (REST) 82

LIST OF FIGURES

Figure 1.1 Screenshots of some popular applications destined for mobile shopping assistance 15
Figure 2.1 The main functions of the MSA from the customer's perspective 18
Figure 2.2 Use Case Diagram 21
Figure 2.3 Add a manager Sequence Diagram 22
Figure 2.4 Add a manager Sequence Diagram 22
Figure 2.5 Add a Manager sequence Diagram 23
Figure 2.6 Update a product's information Sequence diagram 23
Figure 2.7 Get autherntication Sequence Diagram 23
Figure 2.8 Show a nearby product Sequence Diagram: 24
Figure 2.9 Lookup a product's information Sequence Diagram 24
Figure 3.1 Overview of a three-tier application 26
Figure 3.2 Service-Oriented Architecture principles 27
Figure 3.3 Graphical representation of the MVC pattern 29
Figure 3.4 ECB Pattern Example 30
Figure 3.5 Entity-Relationship Diagram 32
Figure 3.6 Relational Schema 33
Figure 3.7 Mobile Client Navigation Diagram 35
Figure 3.8 Web Client Navigation Diagram 36
Figure 3.9 Web Application - Class Diagram 37
Figure 3.10 Mobile Application - Class Diagram 38
Figure 3.11 Lookup Product Information Detailed Sequence Diagram 39
Figure 3.12 Scan a Product's Barcode - Detailed Sequence Diagram 39
Figure 3.13 Show Nearby Products - Detailed Sequence Diagram 40
Figure 3.14 Add a manager - Detailed Sequence Diagram 40
Figure 3.15 Add a Product - Detailed Sequence Diagram 41
Figure 3.16 Update Product Information – Detailed Sequence Diagram 41
Figure 4.1 Android Architecture and major components 45
Figure 4.2 Implementation from the clients and server perspective 50
Figure 4.3 Main Menu Android Interface 52
Figure 4.4 Loading Products list Android interface 53
Figure 4.5 Viewing products list Android Interface 53
Figure 4.6 Look Up Product Info Android Interface 54
Figure 4.7 Manual Product Search Android Interface 54
Figure 4.8 Unfound data Android Interface 55
Figure 4.9 Scan product barcode Android Interface 56
Figure 4.10 Rate product Android Interface 56

Figure 4.11 Show Map Android Interface 57
Figure 4.12 Zoomed in Map Area tag Android interface 58
Figure 4.13 Wi-Fi not enabled Android Interface 58
Figure 4.14 Screenshot of the web application welcome page 59
Figure 4.15 Screenshot of the web application login page 60
Figure 4.16 SQL code of the user 'Caty' insertion into the database 60
Figure 4.17 Example of false log in mode 60
Figure 4.18 Screenshot of the database administrator profile 61
Figure 4.19 Screenshot: Add a manager 61
Figure 4.20 Screenshot: Delete a manager 62
Figure 4.21 Screenshot: View managers 62
Figure 4.22 Screenshot: Add a product 63
Figure 4.23 Screenshot: Delete a product 63
Figure 4.24 Screenshot: View products 64
Figure 4.25 Barcode relative to 6191577600015 64
Figure 4.26 Screenshot: Update a product 65
Figure 4.27 Screenshot: Database up to date 65
Figure 4.28 Screenshot: Web application top banner 65
Figure 4.29 Screenshot: Login 66
Figure 4.30 Screenshot: Add a product 66
Figure 4.31 Screenshot: Database up to date 66
Figure 4.32 Screenshot: Update a product 67
Figure 4.33 Screenshot: View products 67
Figure 4.34 Project Timeline 68
Figure A.1 GPS approach for indoor-positioning 75
Figure A.2 RFID tagging technique 75
Figure A.3 Indoor positioning using 3G mobile telecommunications 76
Figure A.4 Using Bluetooth as an Indoor-Positioning technique 77
Figure B.1 our customized virtual shopping mall map designed with Photoshop 79
Figure B.2 Our demo map after rendering with the Map Studio 80
Figure B.3 Wi-Fi positioning using triangulation technique 81
Figure B.4 Screenshots of the Map Viewer and Map Wifi Client 81

LIST OF TABLES

Table 4.1 Comparison between WP7, IOS and Android *44*
Table A.1 Common RSSI to RSS conversions *78*

ACRONYMS

M.S.A.: Mobile Shopping Assistant

G.P.S.: Global Positioning System

A.P.I.: Applications Programming Interface

R.E.S.T: Representational State Transfer

U.M.L..: Unified Modeling Language

S.O.A.: Services Oriented Architecture

S.O.A.P.: Simple Object Access Protocol

W.S.D.L.: Web Services Description Language

U.D.D.I.: Universal Description, Discovery and Integration

A.P.: Access Point

S.S.ID.: Service Set Identifier

R.S.S.I.: Received Signal Strength Indication

B.S.S.ID.: Basic Service Set Identification

JSON: JavaScript Object Notation

WLAN: Wireless Local Area Network

E.B.C.: Entity Boundary Control

M.V.C.: Model View Controller

E.D.A.: Event Driven Architecture

P.H.P.: Hypertext Preprocessor

X.M.L: Extensible Markup Language

H.T.M.L.: HyperText Markup Language

S.Q.L.: Structured Query Language

GENERAL INTRODUCTION

Throughout the recent years, mobile application development in the world has known a significant increase. Possessing a Smartphone has nowadays become a necessity whether it is for the developers or for the consumers of these apps.
As a matter of fact, 24% of the telephones sold between January and March 2011 are Smartphones. There were 500 million of Smartphones holders in the world in 2010. They should be around 800 million in 2015. Major supporting Operating Systems are Android, IOS, and Windows Phone 7. 36% of Smartphones are under Android, against 26% for Apple in the USA in April 2011.

Among the Smartphones utilities, shopping assistance is undeniably a powerful tool that developers have had quite a thrill trying to code in different ways and for different objectives. That's why supermarkets are orienting their marketing strategies towards investing in this field even more, especially with the ever-increasing number of Smartphone-holding consumers. Their target would be to facilitate the shopping experience for their clients and to market their products for a wider range of consumers through innovative mobile applications.

Our Design &Development Project fits in with this context. The flow of our report will be as following: We will start by presenting the state of the Art: introducing the themes that we are going to be dealing with and a background analysis. The second part will be all about the requirement specifications, first in a general approach and then in detail. After that, we are going to explain the conception of the project, commonly known as the design phase, as we will expose the different layers and how they interact within the architecture. Last but not least, will be the part where we show the project implementation by selecting screenshots of the user interface.

Chapter 1 PROJECT PRESENTATION

The project consists in conceiving an innovative application that embodies the concepts of mobile marketing, indoor positioning and web services. It is destined to guide the shopper through his journey inside the mall by offering shopping assistance location-based services. As the application is going to be built on a mobile platform, we will start off by introducing general concepts related to the notions of mobility, context-aware services and indoor positioning. We will then present how a mobile application works, the different layers it encompasses along with the definition of web services and their major utilities.
On a second note, we will give an insight of what mobile marketing has got to offer today, through our background analysis commonly known as State of the Art.

1.1 General Concepts

1.1.1 Mobility

The concept of mobility has appeared a long time ago with the means of transport. It means the ability to move and be in motion. During the last few years, this concept has evolved in a significant way, especially with the advances of mobile technologies. Mobility can be defined as: the application of mobile devices and wireless technology to enable communication, information access and business transactions from any device, from anyone, from anywhere, at any time.

1.1.2 Location-awareness

Location awareness is a component of presence technology that delivers information about a device's physical location to another user or application. The term is most often used in reference to mobile communication devices and cameras but it can also refer to websites that request a user's zip code to deliver targeted information.

A device's location is usually determined by one of three methods: by GPS satellite tracking, by cellular tower triangulation, or by the device's media access control (MAC) address on a Wi-Fi network.

Location awareness is a growing trend in hardware and software. Here are a few of the current applications: Camera memory cards that automatically tag the location of a picture, Application programs (apps) on smartphones., GPS systems in vehicles, Supply chain management (SCM), Healthcare device management…

1.1.3 Location-based Services

A location-based service (LBS) is a software application for an IP-capable mobile device that requires knowledge about where the mobile device is located. Location-based services can be query-based and provide the end user with useful information such as "Where is the nearest ATM?" or they can be push-based and deliver coupons or other marketing information to customers who are in a specific geographical area.

An LBS requires five basic components: the service provider's software application, a mobile network to transmit data and requests for service, a content provider to supply the end user with geo-specific information, a positioning component and the end user's mobile device. By law, location-based services must be permission-based. That means that the end user must opt-in to the service in order to use it. In most cases, this means installing the LBS application and accepting a request to allow the service to know the device's location.

Location-based services (LBS) are offered by some cell phone networks as a way to send custom advertising and other information to cell-phone subscribers based on their current location. The cell-phone service provider gets the location from a GPS chip built into the phone, or using radiolocation and trilateration based on the signal-strength of the closest cell-phone towers (for phones without GPS features).

Although location-based services have been around since 2000, they have mostly been used in commerce with a subscription-based business model. The release of Apple's 3G iPhone and Google's LBS-enabled Android operating system, however, has allowed developers to introduce millions of consumers to LBS. According to the 2008 fourth-quarter report from Nielsen Mobile, a division of The Nielsen Company, location-based services account for 58 percent of the total downloaded application revenue for mobile phones in North America.

1.1.4 Mobile Application Definition

A mobile application, also known as mobile app, is a term used to describe software that runs on smartphones and mobile phones. Mobile applications are designed to educate, entertain, or assist consumers in their daily lives.

1.1.5 Mobile Application Development

Mobile application development is the set of processes and procedures involved in writing software for small, wireless computing devices such as smartphones or tablets.

Mobile application development is similar to Web application development and has its roots in more traditional software development. One critical difference, however, is that mobile applications (apps) are often written specifically to take advantage of the unique features a particular mobile device offers. For instance, a gaming app might be written to take advantage of the iPhone's accelerometer.

1.1.6 Mobile Marketing

"The starting point has to be consumers. […] This has huge implications for brands. Consumers now decide in a flash who to push up and who to push down. […]
Mobile phones are an inextricable part of this new reality. They are the ultimate device that connects people, to each other, to entertainment and to brands." *says* ***Andrew Jones, Managing Director, Aerodeon***. [N3]

"Mobile Marketing offers the most direct, most personal way of getting in touch with your customers. Whether we are talking awareness, sales promotion or customer retention, Mobile Marketing adds an interactive dimension that is unparalleled for effectiveness and immediacy. " *says* **Emma Cotton, Managing Director, 12snap UK**. [N3]

Mobile marketing can refer to one of two categories of interest. First, and relatively new, is meant to describe marketing on or with a mobile device, such as a cell phone using SMS Marketing.

Second, and a more traditional definition, is meant to describe marketing in a moving fashion - for example - technology road shows or moving billboards.

Although there are various definitions for the concept of mobile marketing, no commonly accepted definition exists. Mobile marketing is broadly defined as "the use of the mobile medium as a means of marketing communication" or "distribution of any kind of promotional

or advertising messages to customer through wireless networks". More specific definition is the following: "using interactive wireless media to provide customers with time and location sensitive, personalized information that promotes goods, services and ideas, thereby generating value for all stakeholders"

Mobile marketing is commonly known as wireless marketing. However wireless is not necessarily mobile. For instance, a consumer's communications with a Web site from a desktop computer at home, with signals carried over a wireless local area network (WLAN) or over a satellite network, would qualify as wireless but not mobile communications. [HTTP://EN.WIKIPEDIA.ORG/]

Mobile marketing has several definitions. Some of them do intercept the same framework and others might diverge to hazy explanations. However, what we've held from all these definitions is that Mobile marketing is a new trendy way of interacting between shopping mall holders and their customers. It enables them to attract people through applications on their mobile phones or on their personal computer web browser. It makes the shopping experience more enjoyable and fruitful for both of the shopping mall manager and the client.

1.1.7 Web Services

A Web service is a method of communication between two electronic devices over the web (internet). The W3C defines a "Web service" as "a software system designed to support interoperable machine-to-machine interaction over a network". It has an interface described in a machine-processable format (specifically Web Services Description Language, known by the acronym WSDL). Other systems interact with the Web service in a manner prescribed by its description using SOAP messages, typically conveyed using HTTP with an XML serialization in conjunction with other Web-related standards."

The term Web services describes a standardized way of integrating Web-based applications using the XML, SOAP, WSDL and UDDI open standards over an Internet protocol backbone. XML is used to tag the data, SOAP is used to transfer the data, WSDL is used for describing the services available and UDDI is used for listing what services are available. Used primarily as a means for businesses to communicate with each other and with clients, Web services allow organizations to communicate data without intimate knowledge of each other's IT systems behind the firewall

Unlike traditional client/server models, such as a Web server/Web page system, Web services do not provide the user with a GUI (graphical User Interface). Web services instead share business logic, data and processes through a programmatic interface across a network. The applications interface, not the users. Developers can then add the Web service to a GUI (such as a Web page or an executable program) to offer specific functionality to users.
Web services allow different applications from different sources to communicate with each other without time-consuming custom coding, and because all communication is in XML, Web services are not tied to any one operating system or programming language. For example, Java can talk with Perl, Windows applications can talk with UNIX applications.

Web services can also be used to implement architecture according to service-oriented architecture (SOA) concepts, where the basic unit of communication is a message, rather than an operation. This is often referred to as "message-oriented" services.

1.2 State of the Art

Through the extensive research we have found out that multiple applications built around the shopping theme do actually exist on the market. The services they offer vary from simple barcode scanning to more sophisticated levels of price-comparison, navigation solutions, virtual shopping carts and mobile payment.

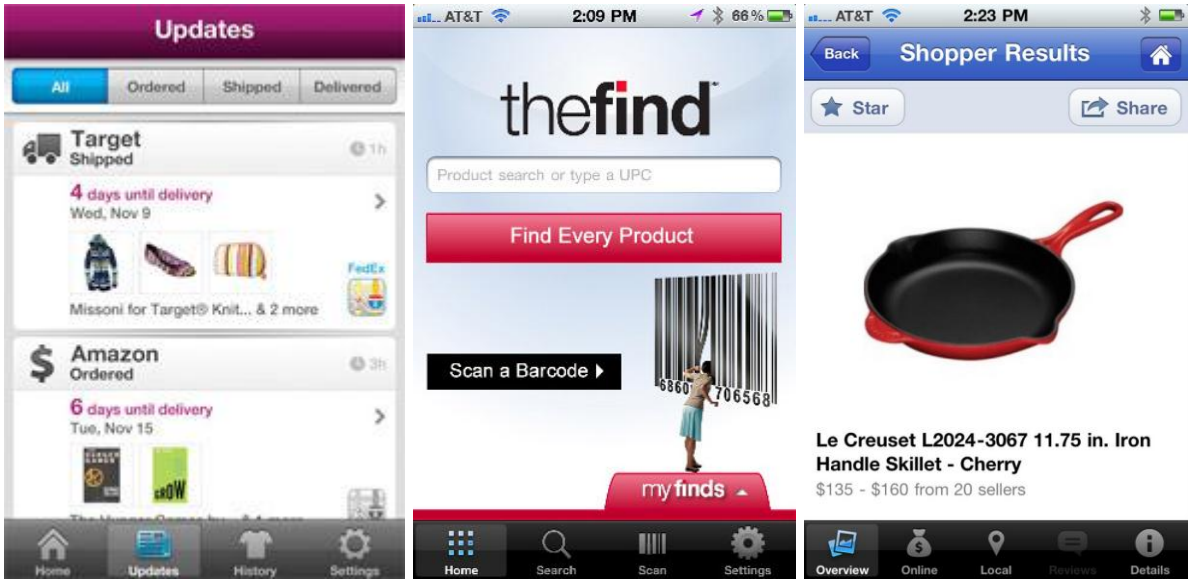


Figure 1.1 Screenshots of some popular applications destined for mobile shopping assistance

Most of these applications do not however include location-aware services, as the indoor-positioning is still one of the main problems for the developers of such applications. Locating the customer while he shops through the aisles, is not as simple as it may seem.

Signals used by outdoor locating technologies for geo-localization are often inadequate in this setting.

In order to proceed with our development we had to check out the different techniques that already exist before casting a choice on a particular technology. A more detailed study can be found in our Annex.

1.3 Conclusion

As we have seen through the State of the Art, many shopping-centered applications already exist on the market but most of them do not support indoor –based services. Building on this fact, we will attempt through our solution development to come up with new innovative ways to make up for those missing parts.

Having defined the main concepts revolving around mobility, location-based services and mobile marketing, all of which are useful for the elaboration of our project, we will present in the next chapter the specification and analysis of the requirements in order to discern the functionalities of the desired solution we aim to set up.

Chapter 2 REQUIREMENTS SPECIFICATION

Through this chapter, we are going to specify the requirements of our project. Starting from the functional requirements till getting to the non-functional ones, we are then going to show our project's principal scenarios.

2.1 Functional Requirements:

Functional requirements refer to primary functions that the application being developed must exhibit. The application has to provide a specific set of services according to the nature of the user while meeting some basic requirements like being able to store and retrieve data.

2.1.1 Identifying the Actors

Our application is destined to be used by 3 types of clients, who interact directly with the system.

- Administrator: He is the one in charge of creating and managing the global database of the products. He is also responsible for adding and deleting managers to be able to benefit from the services of our application.
- Shopping Mall Manager: Once registered as a manager by the administrator, any shopping mall manager can setup and maintain their respective products inventory by selecting basic operations like adding, updating or deleting products.
- Customer: The customer is the main user of this application as he is the prime benefactor of all the services our application must provide to assist him through his shopping experience. From the minute he enters the shopping mall, a customer can browse through the products available in his location, manually search for a product, barcode scan a product, navigate through the map and also rate a particular product.

Typically the customer will be considered as The Mobile-client and both the manager and administrator will be addressed as the Web Client in future references or perspectives.



Figure 2.1 The main functions of the MSA from the customer's perspective

The concept, at its very basic, provides for an environment that allows the following:

***for the Administrator**:

- Add products to the global database:

  The administrator is in charge of entering all the data related to any potentially available product in our database.

- Add or delete a manager:

  Since the administrator is the owner of the application, only he can manage the adding or deleting of managers.

***for the Supermarket Manager:**

- Get Authenticated:

  This is a pre-requisite case to validate any other transaction that may occur in the future. To ensure the security and protect the manager's data manipulation, the latter has to log on first.

  A pre-requisite would be: the manager must already be registered.

- Add or delete products:

  Once connected, the manager must have the ability to choose a product already existing in our database, and add it to his selection He can also add specifications relative to the product.

  If he finds the existing of a certain product not relevant anymore, he can of course delete this product.

- Update information about a specific product:

  Another use case for the logged on manager, is to update the information on a specific product, like adding a discount or an enhancement tip.

***for the Customer:**

- Look up Product information :

  The customer searches for a product through our database by entering the product's name while shopping. He will get a page showing information on the desired product.

- View nearby products:

  For this feature, the customer has the possibility to view the products available in his vicinity, particularly in the aisle he is being located at.

  He then has the chance to pick a specific product for a more detailed information viewing.

- Rate a certain product :

  Through this use case, the customer can give an appreciation for a particular product by pressing the "like" or "dislike" button. The consumer community is also aware of the total number of "likes" for a certain product, as this information pops up among the product profile fields.

- View the Shopping Mall Map :

  The client can locate himself on the map and navigate through the map to see the different areas and reference points it contains.

- Scan a product barcode :

  The application must offer the user the possibility to manually scan a desired product's barcode to later view its information

2.2 Nonfunctional Requirements Specifications:

Non-functional requirements refer to several key features that are beyond the purpose of the application, but critical for its operation. These features ensure customer satisfaction and retention.

- Security: the manager can't access his account unless he has been authenticated. He should already be stored in the database prior to any authentication attempt.

- Usability: the user interface must look intuitive, along with an ease of use and adaptability to all sort of customers. It should be organized enough to gather all information related to each product while insuring the readability and clarity of these information.

- Reliability: We have to make sure the server of the application is available at all times and that there is no possibility a breakdown might happen. In case any failure, the application must abort all on-going operations, but must not corrupt any product data or information that is already stored in the database.

- Performance: The application should be as lightweight and efficient as possible, and run without too many hardware demands. Users should be able to complete their tasks within a reasonable amount of time.
- Supportability: The application would currently be limited to the users of Google Android mobile phones, with a later possibility of expansion to other platforms that support external Java applications.

- Deployment: The Supermarket manager interface could be accessed through a website. Whereas the user interface is a mobile application that the customer can download from the Android Market.

2.3 Use case analysis

Following are the use case diagram required for the project, presented from each actor's perspective.

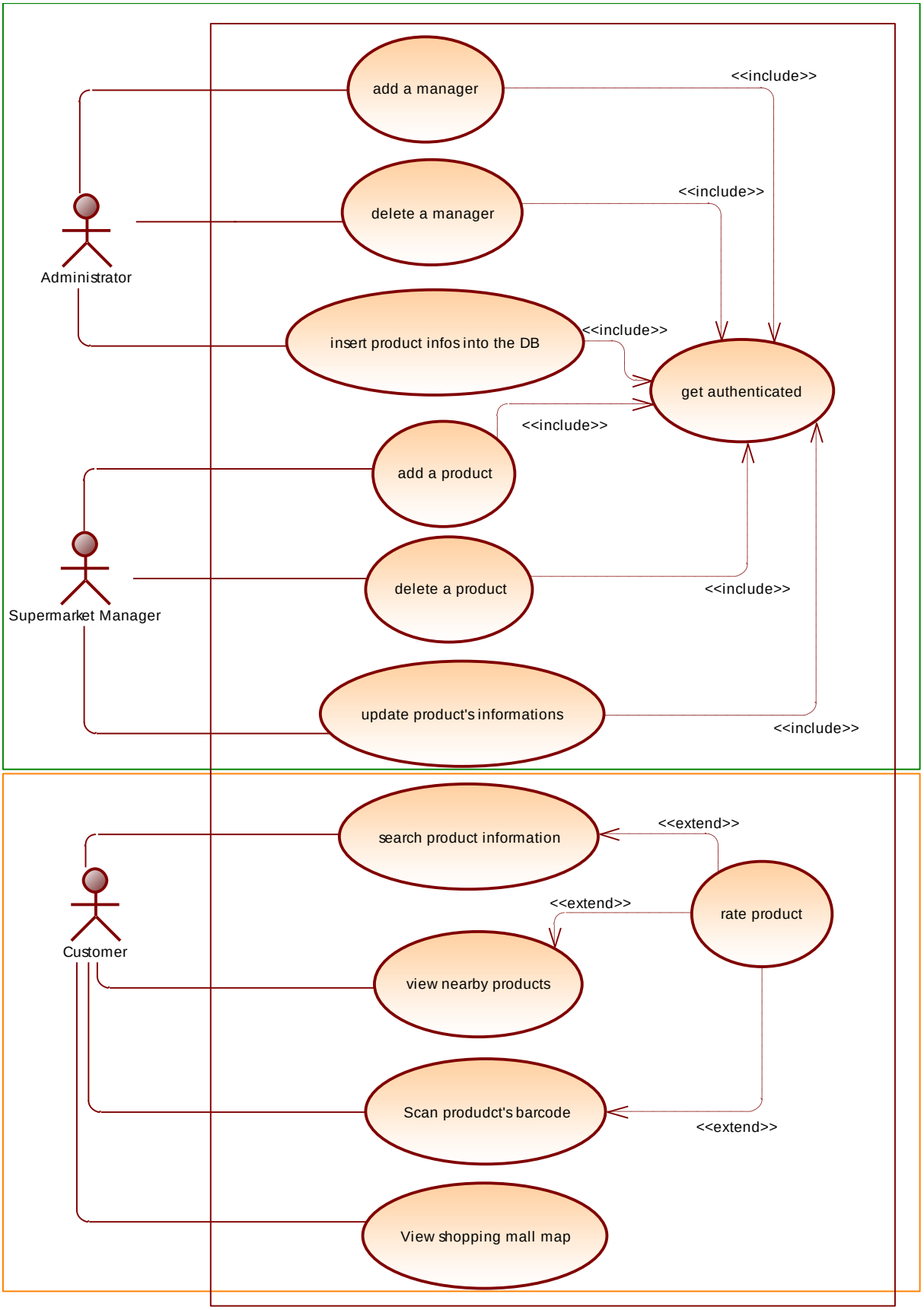


Figure 2.2 Use Case Diagram

2.4 Principal Scenarios

In this section, we are going to show the principal scenarios related to our Mobile Shopping Assistant. We are not going to show all of them. Here are respectively the ones related to the database administrator, the supermarket manager and last but not least the customer.

2.4.1 Database Administrator

2.4.1.1 Authenticate

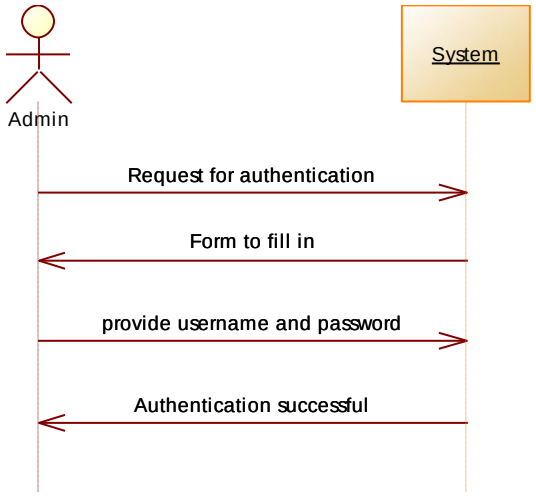


Figure 2.3 Add a manager Sequence Diagram

2.4.1.2 Add a manager

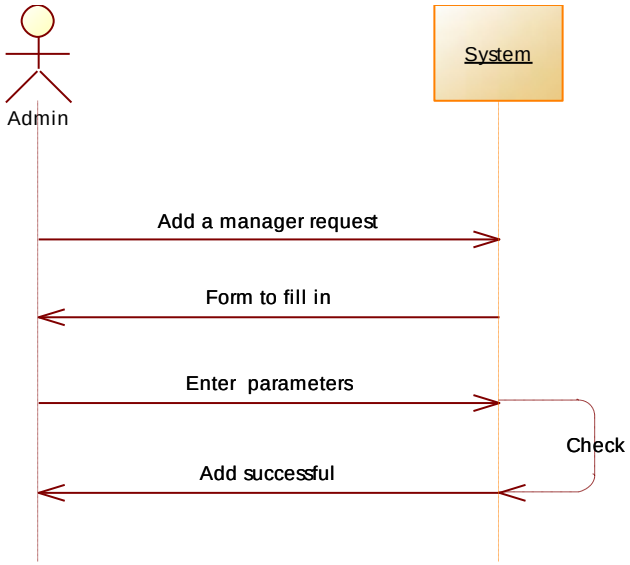


Figure 2.4 Add a manager Sequence Diagram

2.4.2 Supermarket Manager

2.4.2.1 Add a product

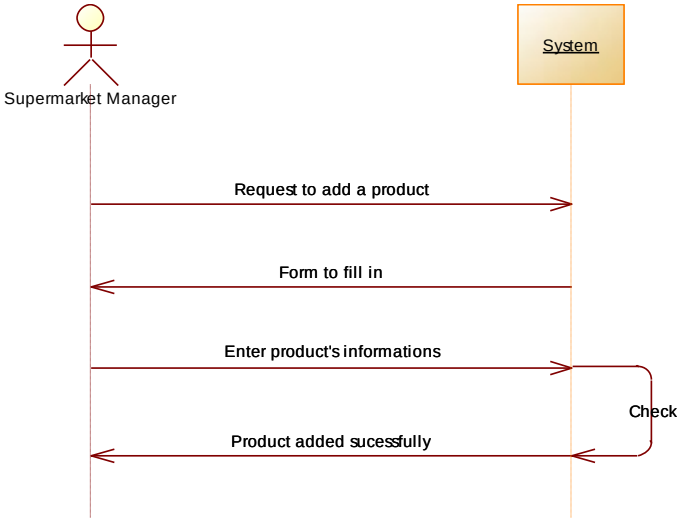


Figure 2.5 Add a Manager sequence Diagram

2.4.2.2 Update product information

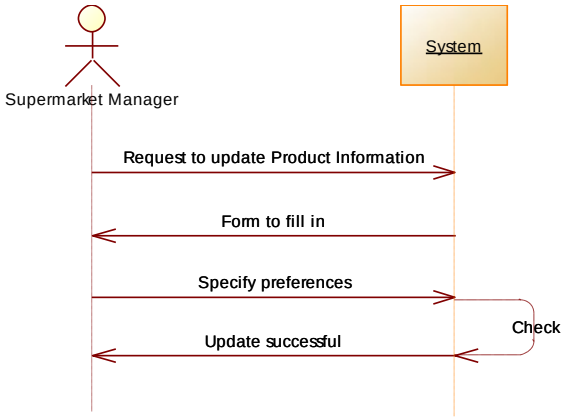


Figure 2.6 Update a product's information Sequence diagram

2.4.2.3 Get Authenticated

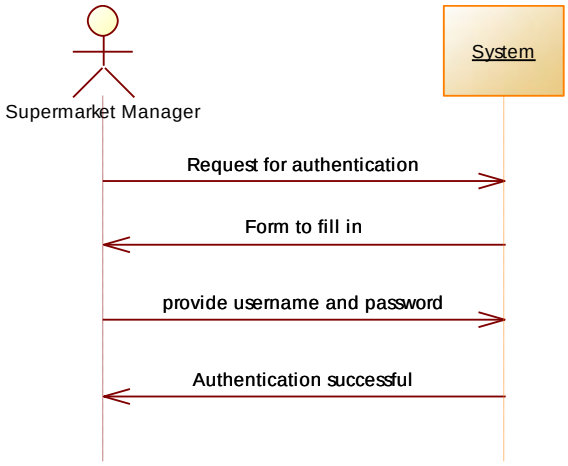


Figure 2.7 Get authernticated Sequence Diagram

2.4.3 Customer

2.4.3.1 Show a nearby product

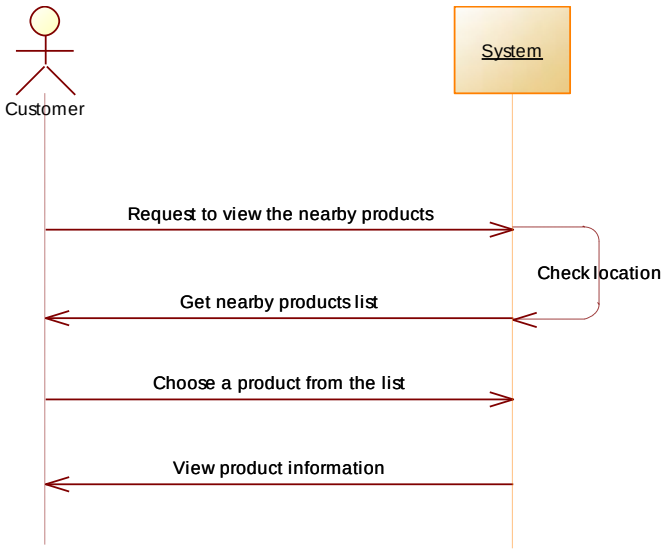


Figure 2.8 Show a nearby product Sequence Diagram:

2.4.3.2 Look up product information

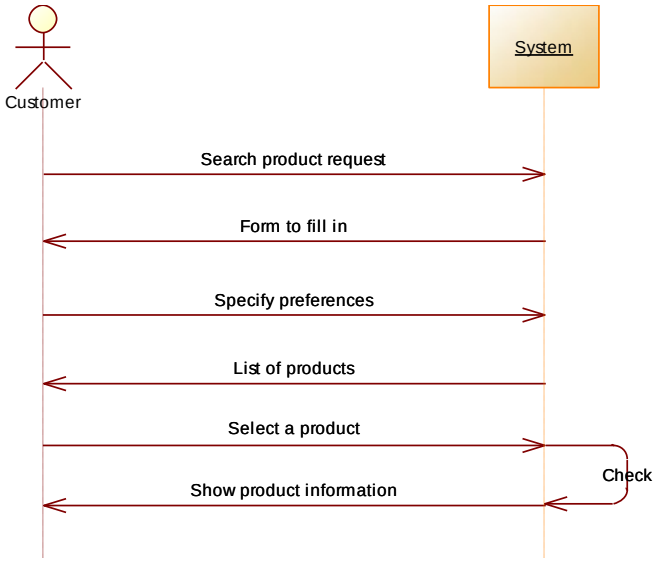


Figure 2.9 Lookup a product's information Sequence Diagram

2.5 Conclusion

In this chapter, we have detailed the functional and nonfunctional requirements of our solution as well as the main actors and the different services it provides while describing each specific use case and its variations with the sequence diagrams. So far we have presented all the functionalities and requirements of the application. In the next chapter the focus will be on how to meet these requirements through the design phase.

Chapter 3 DESIGN

Specifying requirements is indeed crucial to any application development, but what's more important, is knowing how to express and model those requirements in a more eloquent way, commonly known as software engineering.

In this spirit, this chapter aims to represent the different design strategies we applied to extract and highlight the underlying aspects of our application, from the use cases to class diagrams, to architectural structure and so on.

3.1 General Design

3.1.1 Potential Architectures:

Software architecture refers to the composition of the system under development. Every system's configuration is unique due to the stakeholder's desires it supports. Each implementation varies the system's attributes, resulting in a (possible) different architecture.

Some common software architectures are Client-Server, Service-Oriented, Event-Driven, Database Centric, Component-Based, and Distributed Computing.

3.1.1.1 The client-server model

This two-tier architecture communicates between several systems over a network. A client may initiate a communication session, while the server waits for requests from any client (Wikipedia).

3.1.1.2 The multi-tier architecture:

In software engineering, multi-tier architecture (often referred to as n-tier architecture) is a client–server architecture in which the presentation, the application processing, and the data management are logically separate processes. For example, an application that uses middleware to service data requests between a user and a database employs multi-tier architecture. The most widespread use of multi-tier architecture is the three-tier architecture. N-tier application architecture provides a model for developers to create a flexible and reusable application. By breaking up an application into tiers, developers only have to modify or add a specific layer, rather than have to rewrite the entire application over. There should be a presentation tier, a business or data access tier, and a data tier. [Wikipedia]

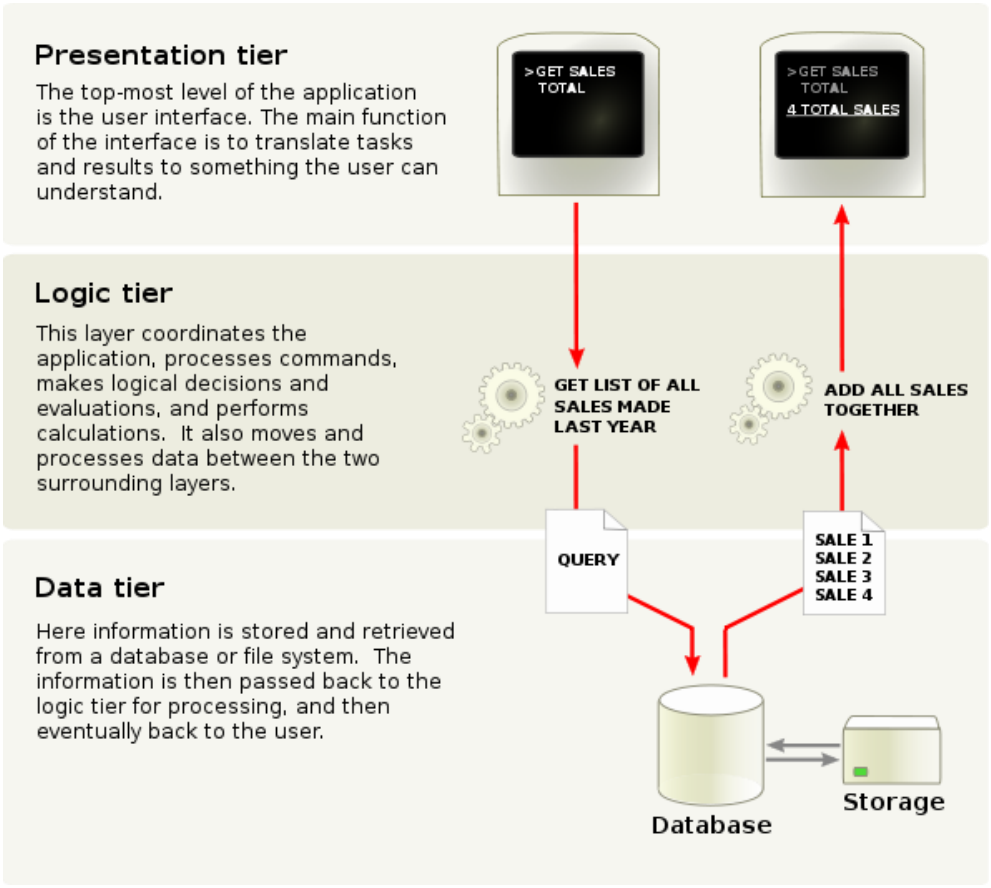


Figure 3.1 Overview of a three-tier application

3.1.1.3 Component based software engineering (CBSE)

It is defined as a type of software architecture style with emphasis on decomposition of the engineered systems into functional or logical components with well-defined interfaces used for communication across the components (Wikipedia).

3.1.1.4 Service-oriented architecture (SOA):

It provides a set of principles of governing concepts used during phases of systems development and integration. Such an architecture will package functionality as interoperable services: software modules provided as a service can be integrated or used by several client interfaces/organization, even if their respective client systems are substantially different.

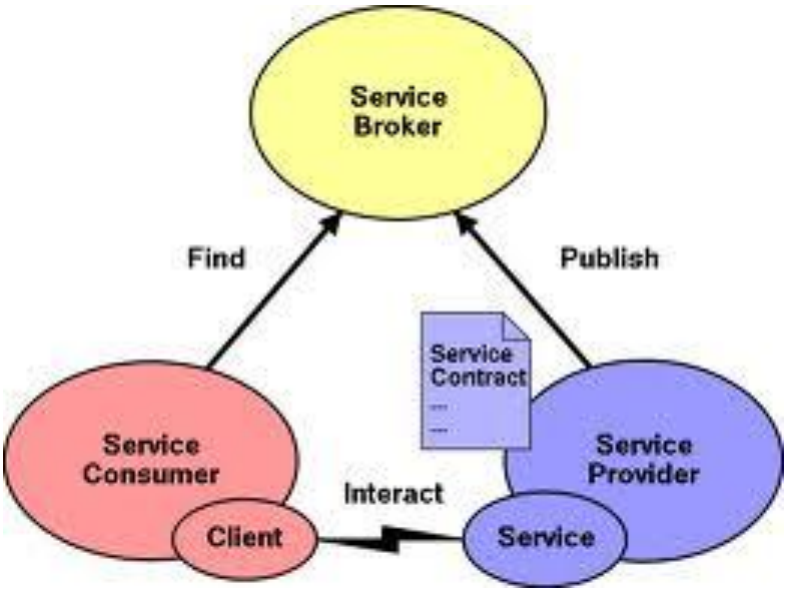


Figure 3.2 Service-Oriented Architecture principles

SOA separates functions into distinct units, or services, which developers make accessible over a network in order to allow users to combine and reuse them in the production of applications. These services and their corresponding consumers communicate with each other by passing data in a well-defined, shared format, or by coordinating an activity between two or more services. (Wikipedia)

3.1.1.5 Event-driven architecture (EDA)

It is a software architecture pattern promoting the production, detection, consumption of, and reaction to events.

Event-driven architecture can complement service-oriented architecture (SOA) because services can be activated by triggers fired on incoming events. This paradigm is particularly useful whenever the sink does not provide any self-contained executive. (Wikipedia)

3.1.1.6 Our Architecture Choice

The Mobile Shopping Assistant is a cross breed of several software architectures. One may question this type of a design method, but there are several reasons why we cannot firmly categorize our system into a single software architectural style. For convenience reasons though, we had to set our records straight about the architecture which our application embodies the most.

3.1.1.6.1 For the Mobile-Client: Service-Oriented Architecture

Since our application is going to invoke web services, the service-oriented architecture choice seemed to fit in the most. Among the various advantages that this architecture provides we can mention the highlights:

- A service-oriented architecture allows us to benefit from all the advantages that client/server architecture has got to offer.
- With its modularity, will be able to easily replace a component (service) with another.
- Even components reusability is possible (as opposed to an all-in-one system made to measures for an organization).
- Better evolution possibilities (We only need to upgrade a service or to add a new one).
- A higher fault-tolerance.
- A facilitated maintenance.

Now that we have clearly stated the final choice for our mobile application architecture, let's move to the web development side.

3.1.1.6.2 For the Web-Client: three-tier Architecture

This time around, the choice was pretty much obvious since it is purely based on the fact that in the web development field, three-tier is often used to refer to websites, commonly electronic commerce websites, which are built using three tiers:

- A front-end web server serving static content, and potentially some cached dynamic content. In web based application, Front End is the content rendered by the browser. The content may be static or generated dynamically.
- A middle dynamic content processing and generation level application server, for example Ruby on Rails, Java EE, ASP.NET, PHP, ColdFusion platform.
- A back-end database or data store, comprising both data sets and the database management system or RDBMS software that manages and provides access to the data.

3.1.2 Potential Architecture Patterns

3.1.2.1 MVC pattern (Model-View Controller)

Model–View–Controller (MVC) is a design pattern for computer user interfaces that divides an application into three areas of responsibility:

the Model: the domain objects or data structures that represent the application's state.

the View, which observes the state and generates output to the users.

the Controller, which translates user input into operations on the model. [Wikipedia]

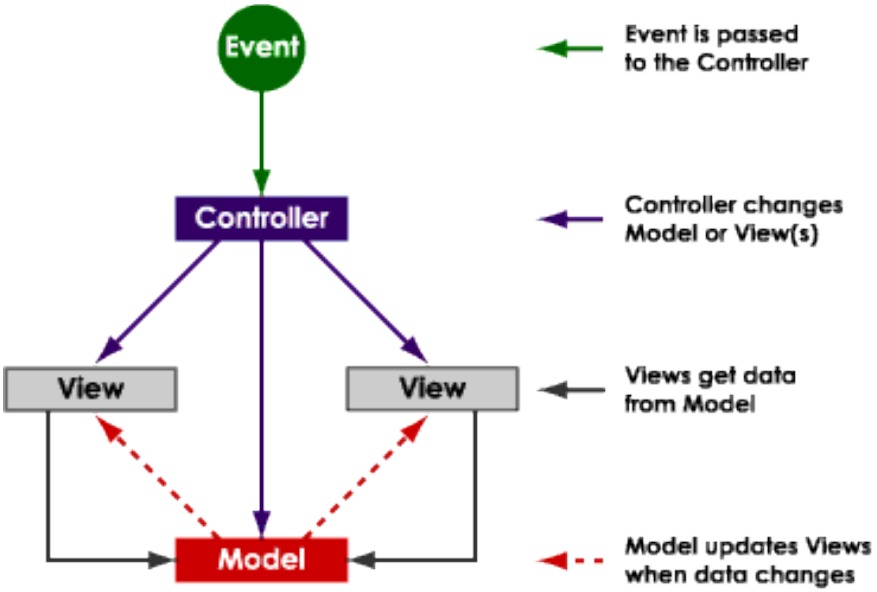


Figure 3.3 Graphical representation of the MVC pattern

Events typically cause a controller to change a model, or view, or both. Whenever a controller changes a model's data or properties, all dependent views are automatically updated. Similarly, whenever a controller changes a view, for example, by revealing areas that were previously hidden, the view gets data from the underlying model to refresh itself.

The model view controller pattern is the most used pattern for today's world web applications. At present there are more than a dozen PHP web frameworks based on MVC pattern.

The MVC pattern separates an application in 3 modules: Model, View and Controller:

The model is responsible to manage the data; it stores and retrieves entities used by an application, usually from a database, and contains the logic implemented by the application.

The view (presentation) is responsible to display the data provided by the model in a specific format. It has a similar usage with the template modules present in some popular web applications, like wordpress, joomla,..

The controller handles the model and view layers to work together. The controller receives a request from the client, invoke the model to perform the requested operations and send the data to the View. The view format the data to be presented to the user, in a web application as an html output.

3.1.2.2 EBC pattern (Entity-Boundary-Control)

The Entity-Control-Boundary Pattern (ECB) is a variation of the Model-View-Controller Pattern. When identifying the elements for some scenario of system behavior, if we can align each participating element with one of three key perspectives: Entity, Control, or Boundary, then our system is more likely to behave according to the EBC pattern.

Entities are objects representing system data: Customer, Transaction, Cart, etc.

Boundaries are objects that interface with system actors: user interfaces, gateways, proxies, etc.

Controllers are objects that mediate between boundaries and entities. They orchestrate the execution of commands coming from the boundary.



Figure 3.4 ECB Pattern Example

3.1.2.3 Our Architecture Pattern choice: MVC

Although we could have worked with EBC pattern, since it's a derivative of MVC, and has a close relationship to our shopping theme, we eventually chose to align ourselves with the MVC pattern. The similarities between our application and the EBC-inspired architecture could only be seen if we considered our data items (products, logins, managers, maps, directions etc.) as entities as, our user interfaces (login interface, scan product interface, add manager interface etc.) as boundaries and the rest of the logic functions and services to be called as controllers (connection service, update product information service, retrieve product information service etc.)

We retained as architecture pattern for our application the Model View Controller pattern as it presents a permanent scenario of communication between the different modules in order to efficiently respond to the clients' requests.

The architecture of our application is divided into three main layers. In fact, this architecture allows a clear separation between the software levels of our application. These three levels are:

- **The presentation layer** : It consists in the visible and user-interactive part, or in other terms : the Human-Machine interface. The presentation layer binds the requests sent by the user to the inferior layer, the business layer, and in return, it resends him the information having been treated by the business layer.

- **The business logic layer:** This layer represents the behavior of the application, the implementation of the logic; it describes the operations on the data and the interactions with the database. It treats the requests sent by the superior layer, the presentation layer. To do so, it relies on the system's data, provided by the services of the inferior layer, the persistence layer. The business logic layer resends in return the treated data to the presentation layer.

- **The persistence layer :** This layer is the one responsible for managing data system access. These data could belong to the system, or coming from other systems. The business layer doesn't make a difference between these various types of data, it accesses them with the same manner. The access to the data is the same no matter the way in which they have been stored. They could simply be put in text files or in a database.

The division of the architecture into three layers ensures the transparency between the different levels, which enables their independence from each other. This abstraction enhances the maintenance of the application.

3.2 Detailed Design

In the following section, we're going to concretize our design through some diagrams using the Unified Modeling Language UML. In order to explain the different aspects of the architecture we've chosen, we begin first of all with showing you our database.

3.2.1 Database administration

We've adopted a clear and simple database so that it is easy enough to retrieve data from when needed. The database can be accessed by anyone of our actors, whether it is the administrator himself, the shopping mall manager or the customer. We will show below here the different relationships between these entities.

3.2.1.1 Entity-Relationship Diagram

The entity-relationship diagram is a conventional diagram that shows the different entities, which our application is composed of and the relationships between each two of them. Here is our entity-relationship diagram:

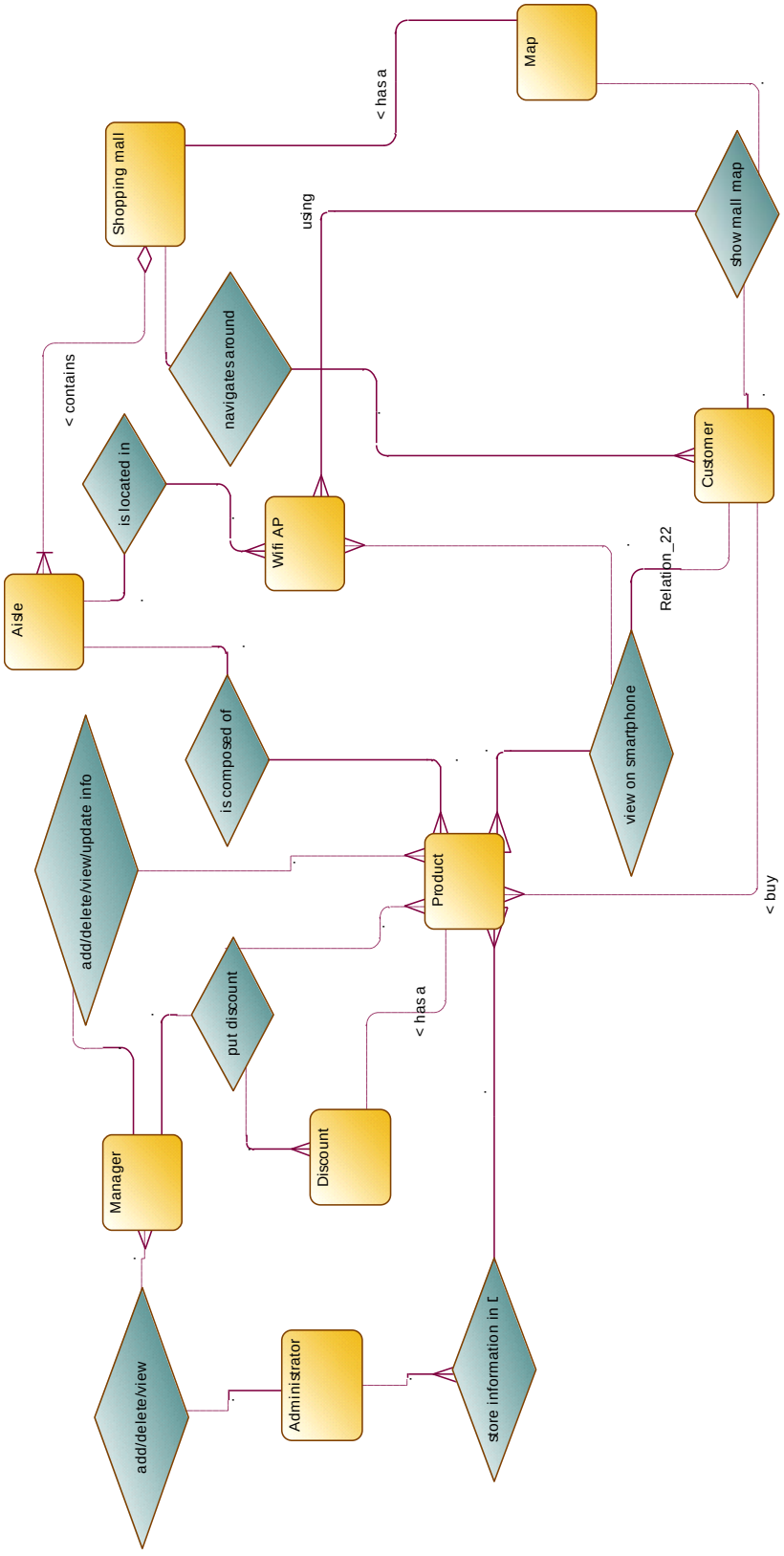


Figure 3.5 Entity-Relationship Diagram

3.2.1.2 Relational Schema

We had to use a database to store our data because of the great amount of information that we have. Hence, we designed the tables shown in figure 3.1 to ensure an easy access and manipulation of our data.

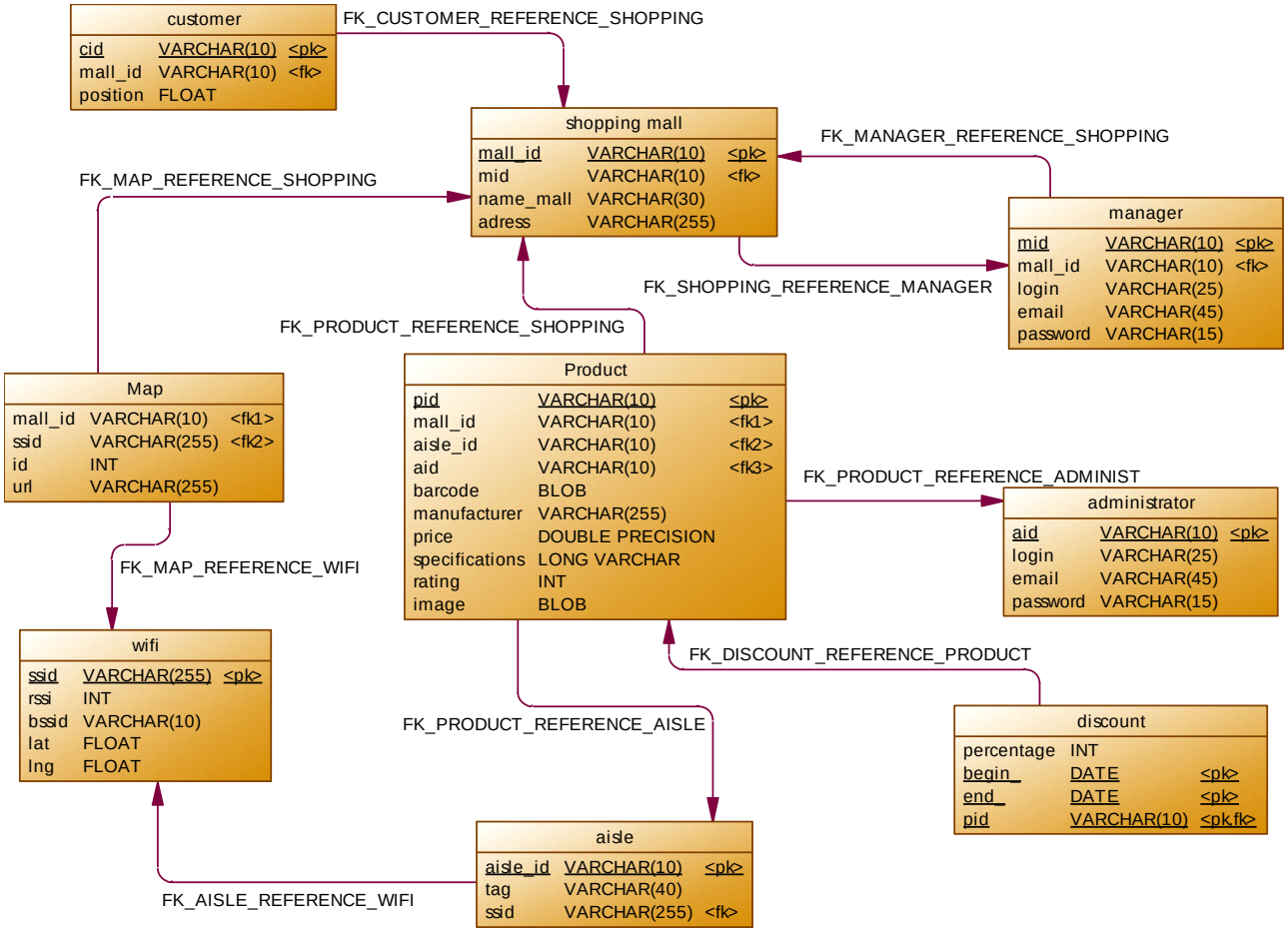


Figure 3.6 Relational Schema

As the above relational schema indicates, we have carefully chosen our database tables to fit the requirements we need to ensure. Here is a brief description of each table used.

- **Product (pid, #mall_id, #aisle_id, #aid, barcode, manufacturer, price, specifications, rating, image).**

The table "product" is our most important table. It includes all the product information needed to be stored such as the price, the barcode, etc. And it references some other tables that define it which are the mall in which the product is sold, the aisle in which is placed and the administrator who added its information in the database.

- **Map (id, #mallid, #ssid, url).**

This table includes the attributes needed to draw the mall map. It references the mall id and the Wi-Fi access point id (ssid). Url is the map url that is going to be retrieved from the web server.

- **Wifi (ssid, rssi, bssid, lat, lng).**

The table "Wi-Fi", meanwhile is related to the Wi-Fi Access Points (AP) we have in the shopping mall. It includes the Wi-Fi APs' specifications which are mainly the SSID, RSSI, BSSID, latitude and longitude of each one.

- **Aisle (aisle_id, tag, #ssid).**

The table "aisle" defines the aisles' principal attributes. We mean by "tag", the aisle's naming. This table references the Wi-Fi table in order to assign each aisle to its corresponding Wi-Fi AP.

- **Discount (begin, end, #pid, percentage).**

The table "discount" includes some discount information such as the discount percentage, it is identified by the begin and end date of the discount period and it references the concerned product's id.

- **Administrator (aid, login, email, password).**

The administrator is the only one who has unlimited access to the database. He is identified with his id (aid) and has a login, an email and a password which allow him to enter whatever information he wants into the database.

- **Manager (mid, #mall_id, login, email, password).**

The manager must be eventually registered by the administrator into our database in order to view and change any product information. He is identified by his id (mid) and has a login, email and password as well. The attribute #mall_id refers to the shopping mall he's working at.

- **Customer (cid, #mall_id, position).**

The customer table includes his id, a reference to the mall he's shopping into and his position in this mall which is calculated considering the Wi-Fi APs positions.

- **Shopping mall (mall_id, #mid, name_mall, address).**

And here we come finally to the shopping mall table which is identified by its id. It contains the id of the manager who is responsible of the products sold there, the name of the shopping mall and its address.

3.2.2 Mobile Client Navigation Diagram

This diagram illustrates the different state transitions that could occur when the client uses the mobile application.

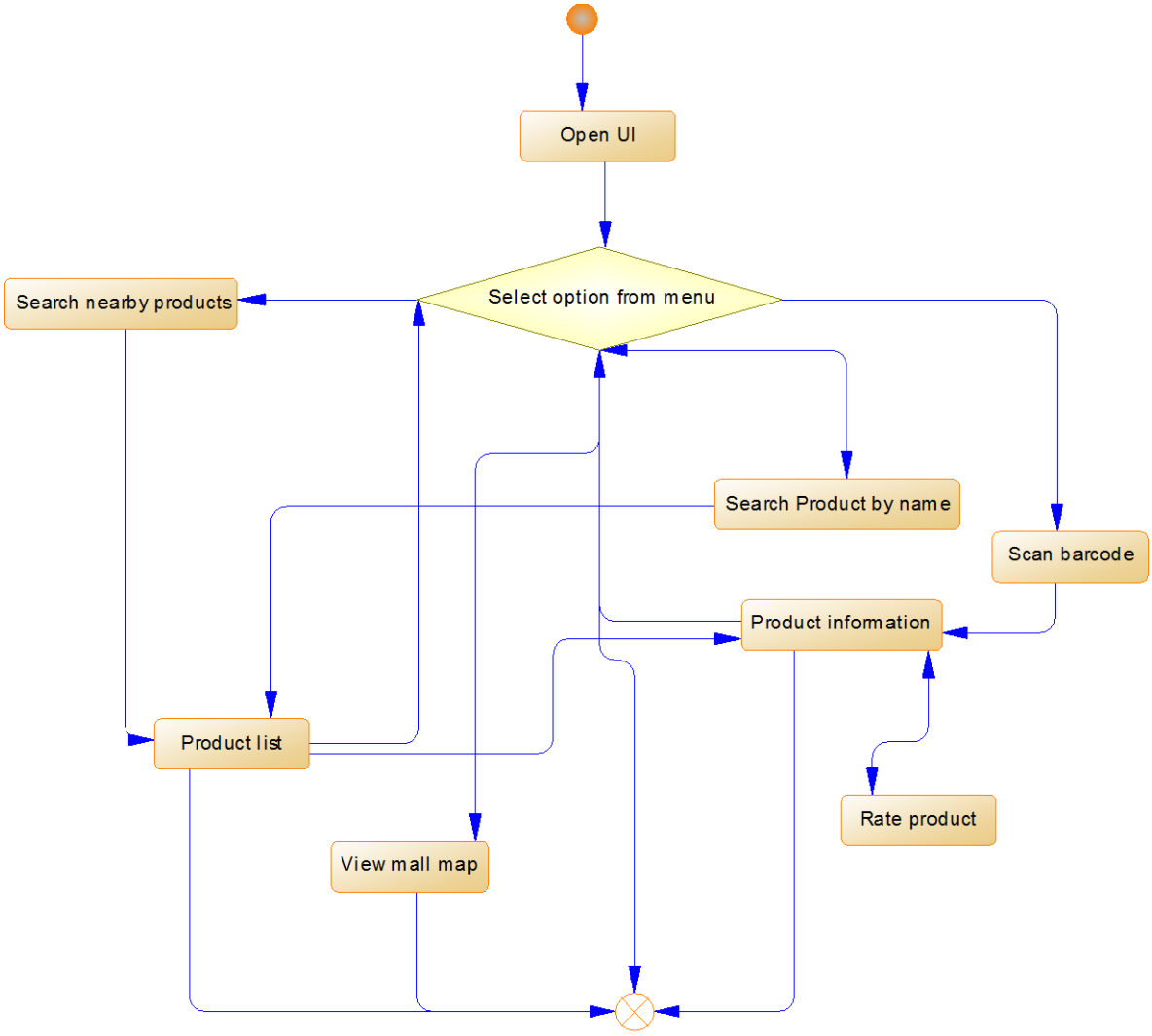


Figure 3.7 Mobile Client Navigation Diagram

3.2.3 Web Client Navigation Diagram

This diagram depicts the various possibilities that the web client user is faced with when navigating through the web application.

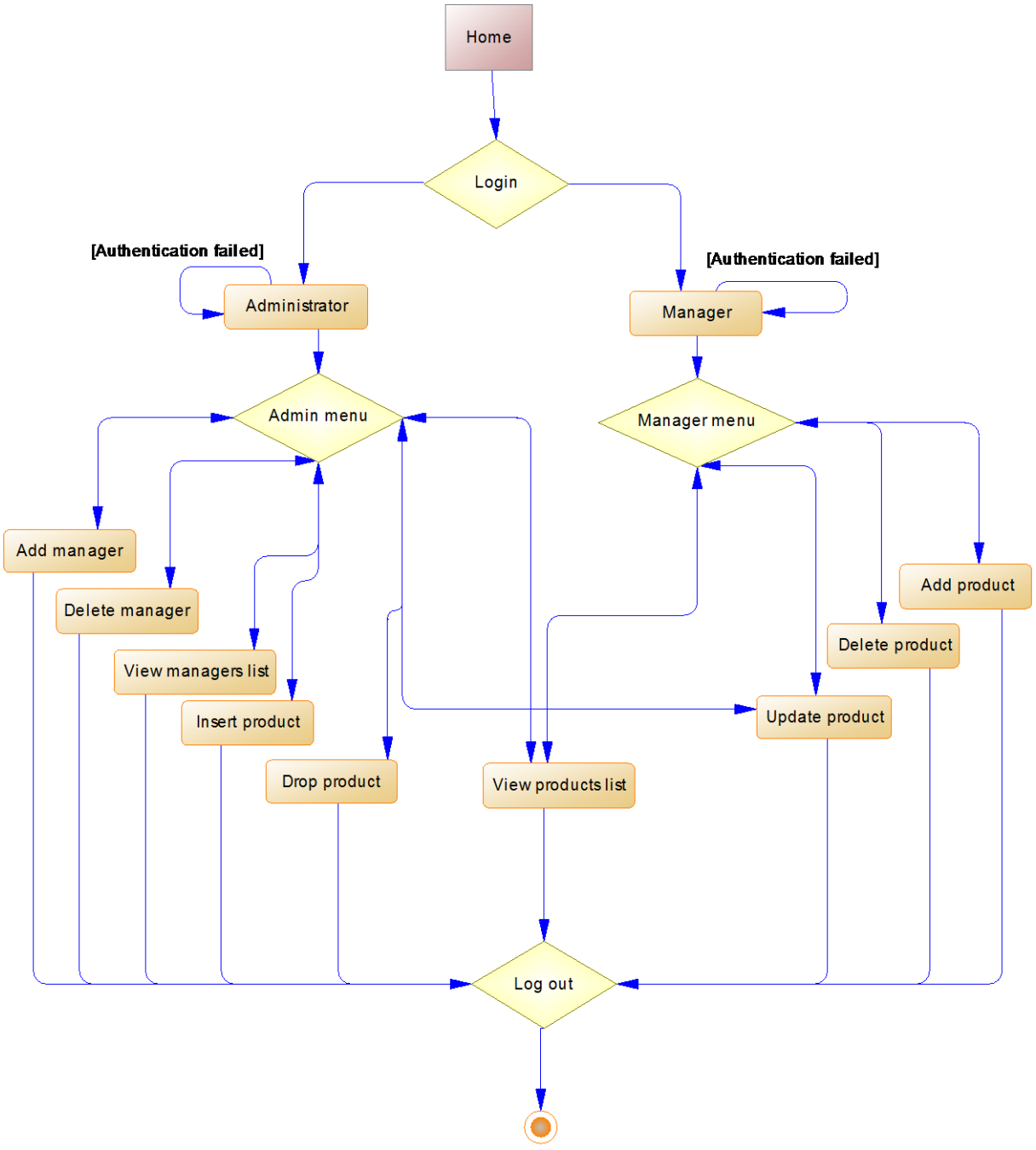


Figure 3.8 Web Client Navigation Diagram

3.2.4 Class Diagram for the Web Application

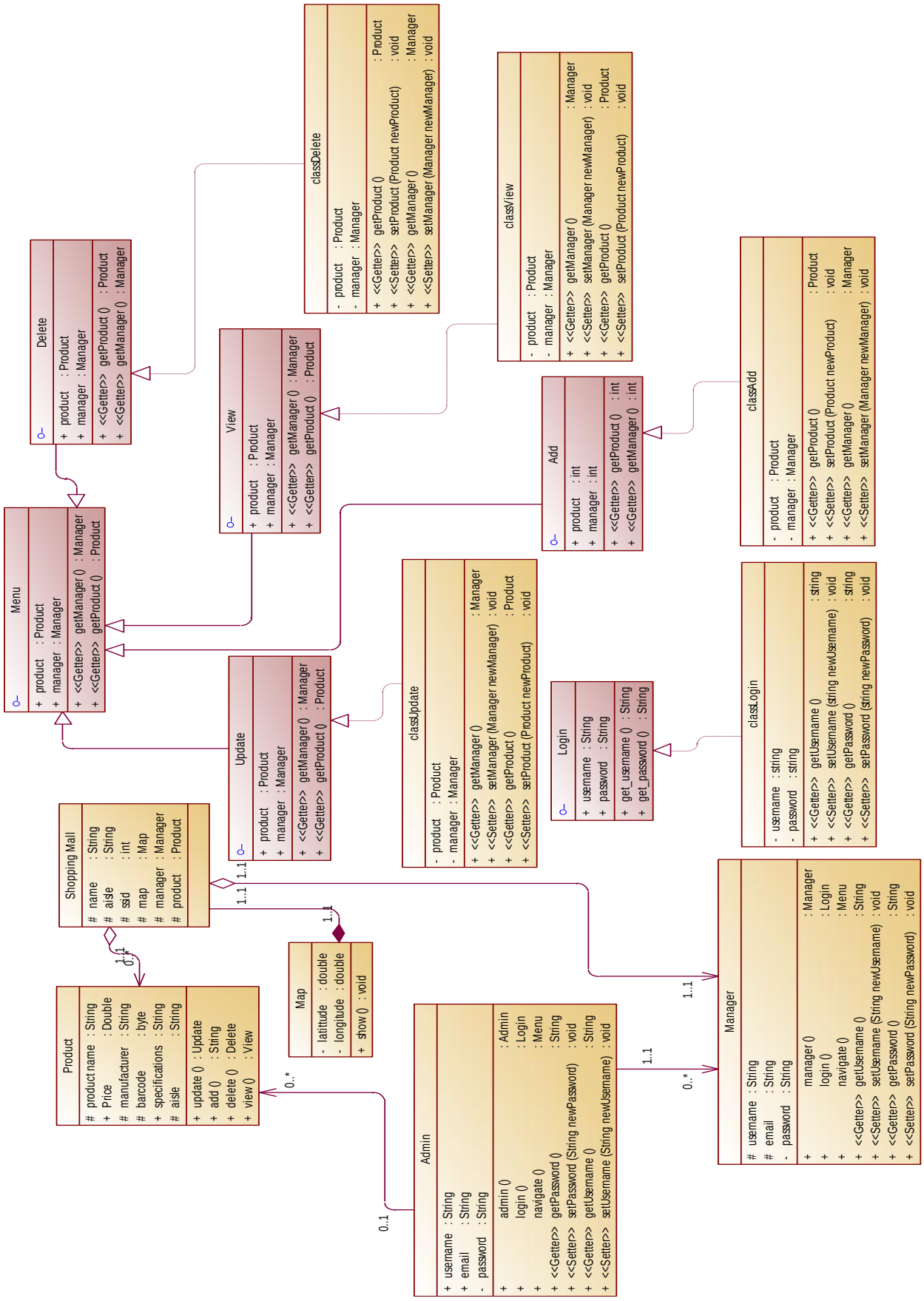


Figure 3.9 Web Application - Class Diagram

3.2.5 Class Diagram for the Mobile Application

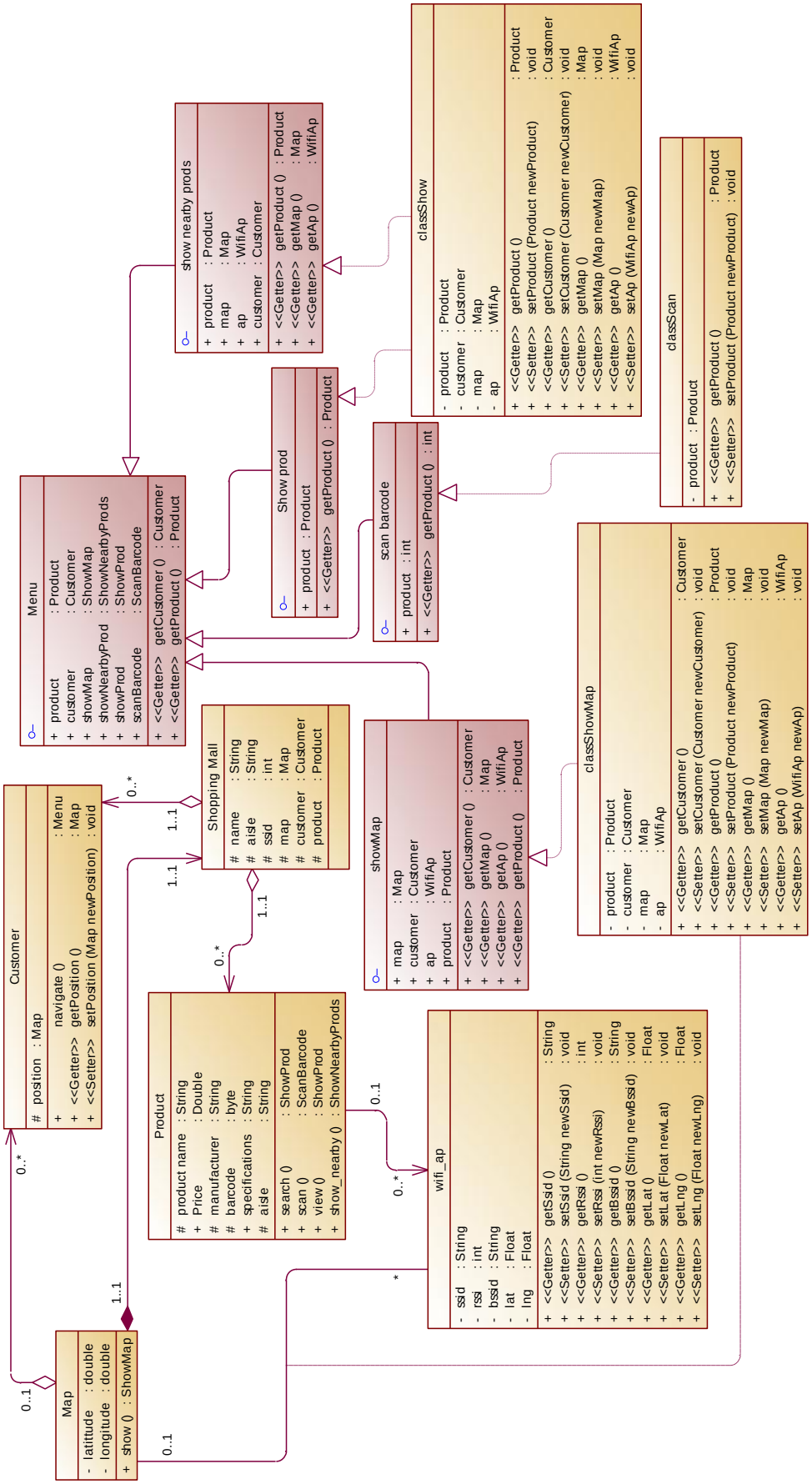


Figure 3.10 Mobile Application - Class Diagram

3.2.6 Detailed scenarios

Below, we have detailed some of the main scenarios of our project. We have studied, for each actor, the possible sequence of events in relation to the concerned classes.

3.2.6.1 For the Customer

3.2.6.1.1 Look up product information

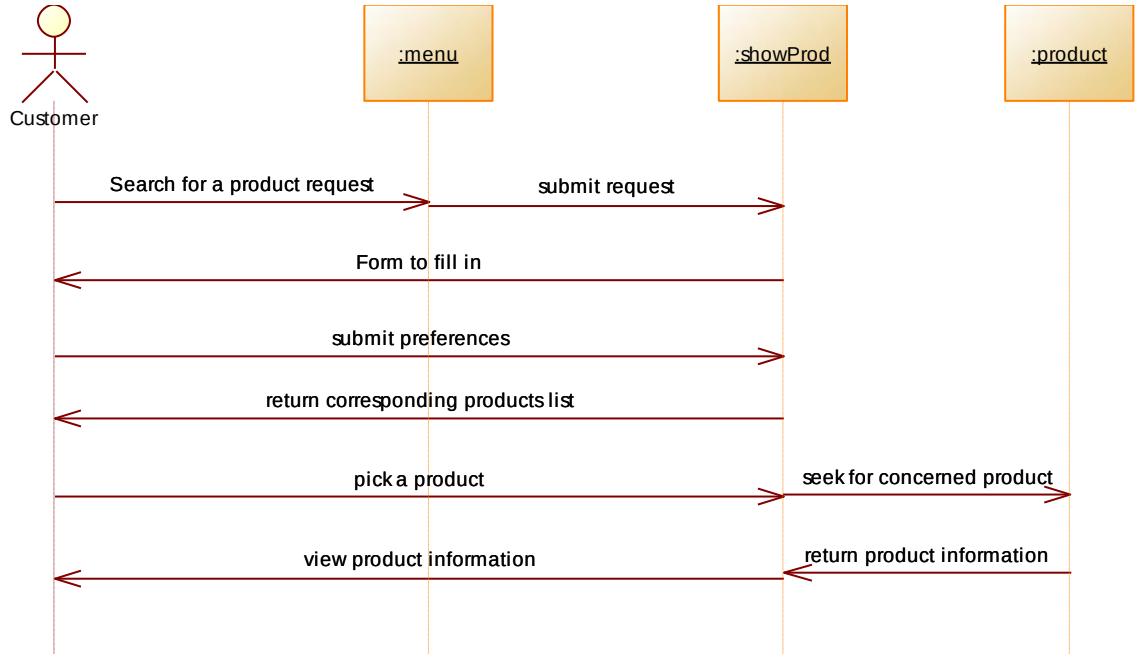


Figure 3.11 Lookup Product Information Detailed Sequence Diagram

3.2.6.1.2 Scan a product Barcode

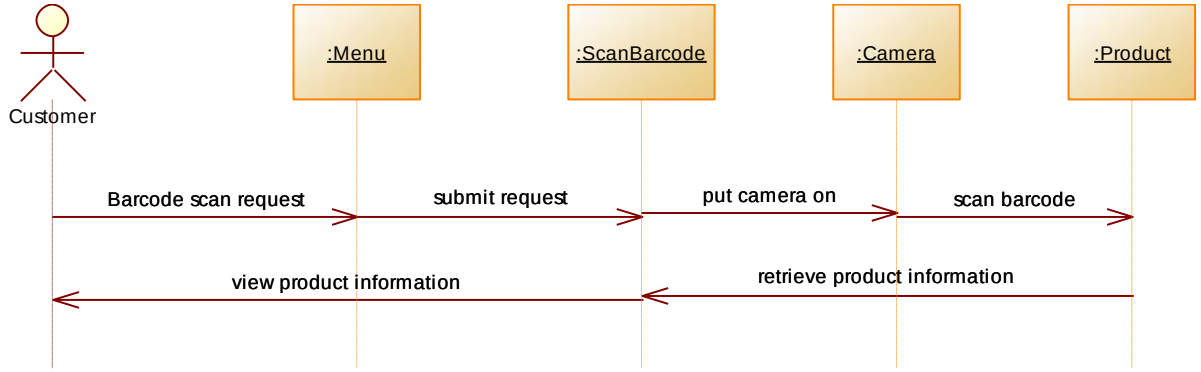


Figure 3.12 Scan a Product's Barcode - Detailed Sequence Diagram

3.2.6.1.3 Show nearby Products

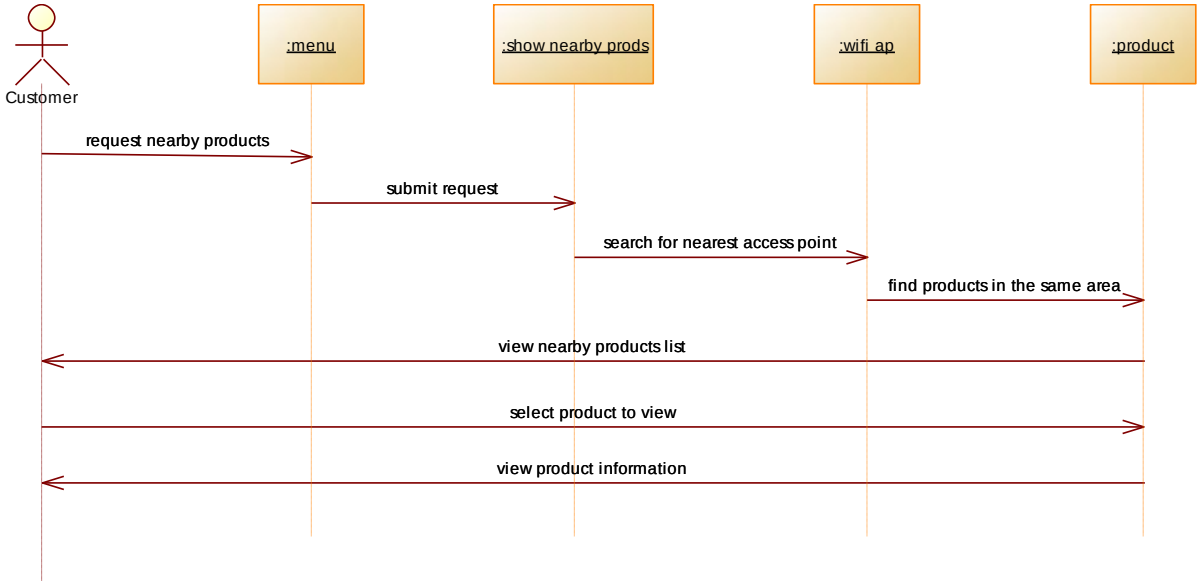


Figure 3.13 Show Nearby Products - Detailed Sequence Diagram

3.2.6.2 For the Administrator

3.2.6.2.1 Add a manager

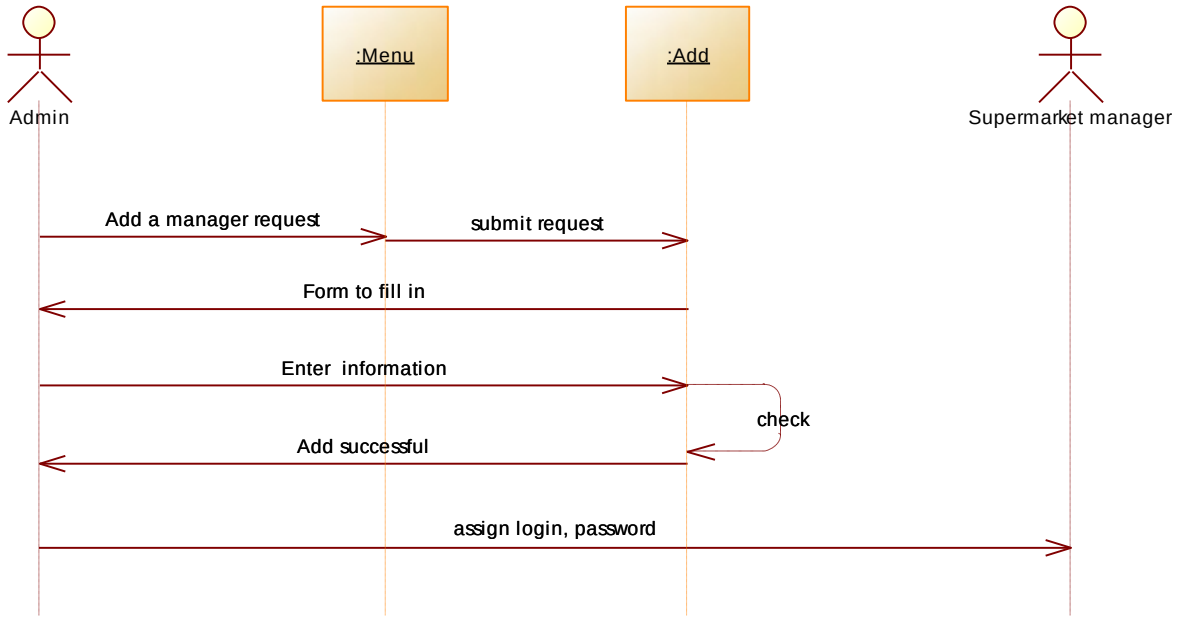


Figure 3.14 Add a manager - Detailed Sequence Diagram

3.2.6.3 For the Supermarket Manager

3.2.6.3.1 Add a product

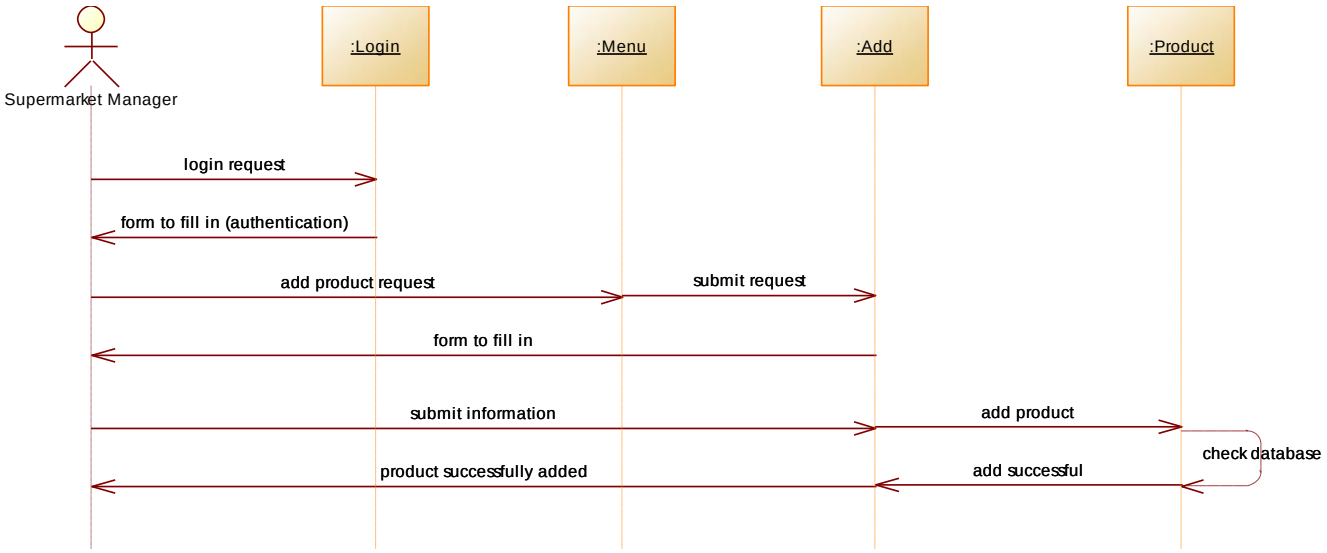


Figure 3.15 Add a Product - Detailed Sequence Diagram

3.2.6.3.2 Update product information

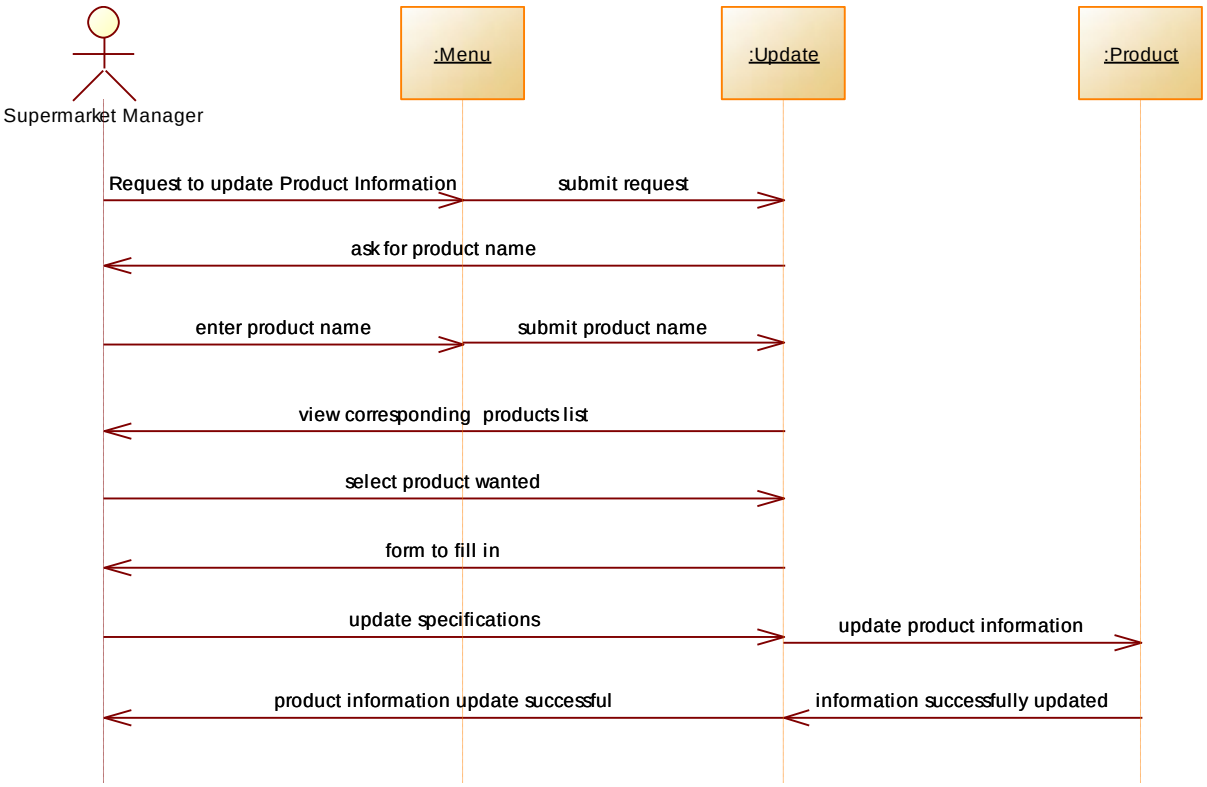


Figure 3.16 Update Product Information – Detailed Sequence Diagram

3.3 Conclusion

All along this chapter we conceived the design of our application by identifying the possible architectures it might embed along with presenting the different UML diagrams and database schemas that we are going to apply for our prototype both from the Mobile client and the Web client perspective. All of this work is in order to prepare for the next stage of our project development which will be presented in the upcoming chapter.

Chapter 4 IMPLEMENTATION

In this chapter we will detail step by step the different aspects of our solution. First we will start off by presenting the hardware environment, the platforms, technologies and softwares that we used in the development of our project. Second, we will show you what our application looks like from each actor perspective, by putting together screen captures of our user interfaces. Last but not least, we will outline the different stages of our project development through a detailed timeline. We will also expose the problems we faced during this development as well as the possible improvements that might be brought in the future.

4.1 Workspace Environment:

We will present in this part the hardware and software environments we used to set up our solution.

4.1.1 Hardware Environment

For the implementation of our project, we worked on 2 personal computers, the configuration of which is as following:

- **First Machine:**
  - Processor Intel® Core™2 Duo CPU P8700 @2.53Ghz 2,53 Ghz
  - RAM 4.00 Go
  - Operating System: Windows Vista™ 64bits
  - Hard Disk: 300 Go
- **Second Machine**
  - Processor Intel® Core™ i5-2450M CPU @ 2.50GHz 2.50GHz
  - RAM 4.00 Go
  - Operating System: Windows 7 Ultimate, SP1, 32bits
  - Hard Disk: 500 Go

4.2 Software Environments and Technological Choices

After having defined our hardware environment, it's time we check out the software environments and justify the technological choices we opted for.

4.2.1 The Client-Side (Front-End): Mobile Development

4.2.1.1 Android

To be able to code in the most efficient ways, the choice of the work environment is very crucial. Since our application is essentially destined for smartphones, the 3 main OS (operating systems) we had to consider were IOS (Apple IPhone), Windows Phone 7 (Microsoft) and Android (Google). After taking a look at some of the statistics, we thought it would be more interesting to make out application on Android as it supports much more features and is exponentially rising on the market. As a matter of fact, Android was listed as the best-selling smartphone platform worldwide in Q4 2010 by Canalys with over 300 million Android devices in use by February 2012.

PCWorld | Windows phone | iOS | android

	Windows Phone 7	iOS (iPhone)	Android
Developer	Microsoft	Apple	Google
Copy/Paste	✘	✔	✔
Multitasking	✘	✔	✔
Flash Support	✘	✘	✔
Silverlight Support	✘	✘	✘
HTML5 Support	✘	✔	✔
Unified Inbox	✘	✔	✔
Exchange Support	✔	✔	✔
Threaded Email	✘	✔	✔
Visual Voicemail	✘	✔	✔
Video Calling	✘	✔	✔ Third Party App
Universal Search	✘	✔	✔
Internet Tethering	✘	✔	✔
Removable Storage	✘	✘	✔
Facebook Integration	✔	✘ (Third Party App)	✔ (Third Party Integration)
Twitter Integration	✘	✘ (Third Party App)	✔ (Third Party Integration)
Folders	Hubs	✔	✔
Apps Organization	Alphabetical	Customizable	Customizable
App Store	1,000+ Apps	300,000+ Apps	90,000+ Apps
Microsoft Office Support	Built-In	Third Party App	Third Party App
Widgets	Tiles on Home Screen	✘	✔
Media Sync	Zune Software Mac & PC	iTunes Mac & PC	Direct File Transfer + Third Party Software
X-Box Live Integration	Built-In	Via Third Party App	Via Third Party App

Table 4.1 Comparison between WP7, IOS and Android

Android is a software stack for mobile devices that includes an operating system, middleware and key applications. The Android SDK provides the tools and APIs necessary to begin developing applications on the Android platform using the Java programming language. [N6]

The following diagram shows the major components of the Android operating system.

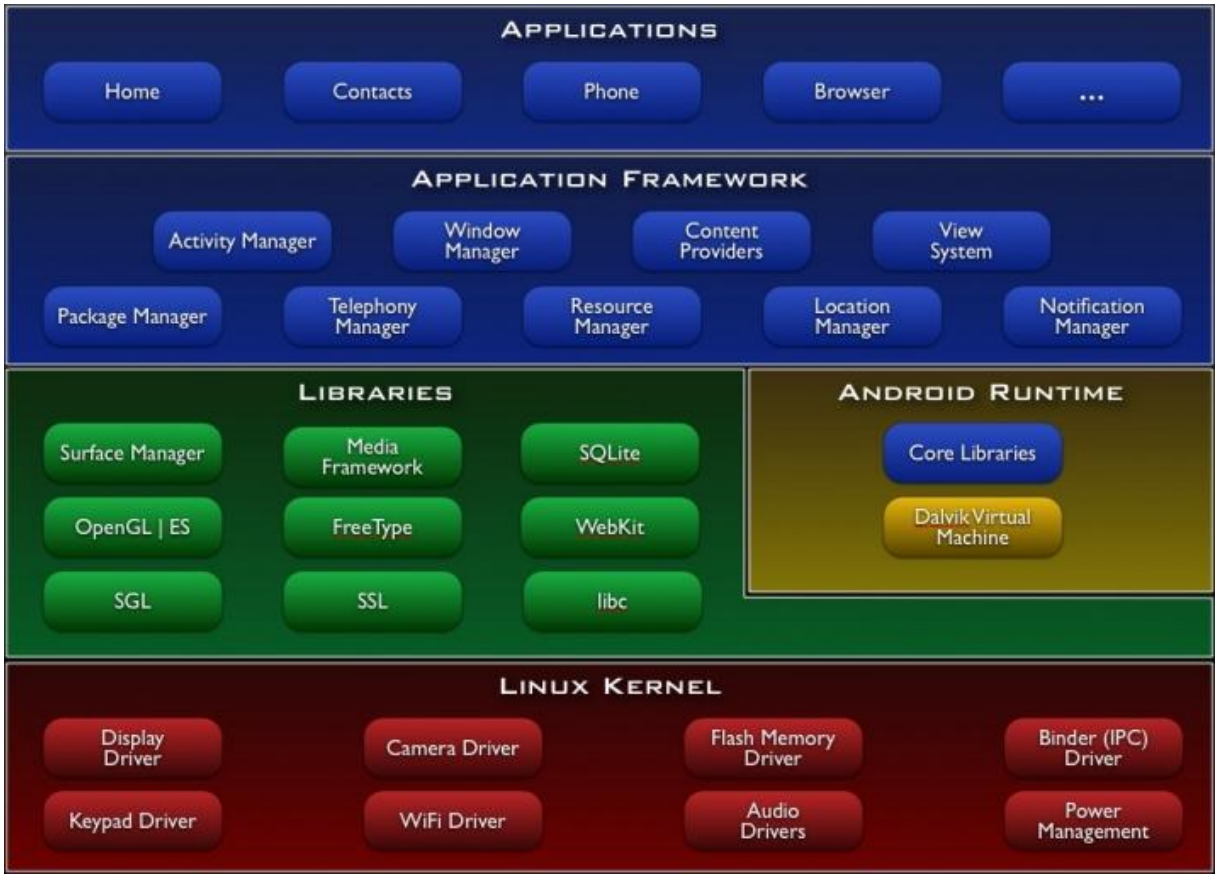


Figure 4.1 Android Architecture and major components

4.2.1.2 Android 2.2 Platform API level 8

As the smartphone development is an ongoing process, the Android platforms are evolving accordingly. Each time, a new feature is added and enhancements are being made on the older versions. In our case, we chose to work on the 2.2 API level 8 version of Android. Even though newer versions do exist, this is the commonly used platform to date. The light-weight virtual machine that comes with it is Dalvik VM which will play the role of the emulator, despite lacking certain features of the classic JVM (Java Virtual Machine).

4.2.1.3 Eclipse as Integrated Development Environment for Android

Having chosen Android, which is JAVA-based, the question now is on which IDE are we going to install the Android SDK (software development Kit) and the ADT (Android Development tools) plugins to start developing our application.

The two possible environments were NetBeans and Eclipse, both being very popular in the development scene and both having very powerful tools.

Since Eclipse is recommended by Google for Android Developers, for the wider range of add-ons and plugins it supports, we ended up choosing Eclipse over NetBeans.

Eclipse is a multi-language software development environment comprising an integrated development environment (IDE) and an extensible plug-in system. It can be used to develop applications in Java and, by means of various plug-ins in other languages as well. A very rich interface allows users to manage their java files and compile programs. The files are structured within a project according to a layout already defined by the various java packages. The integrated syntax analysis makes it easier to understand the code as it highlights the key words in java files. Eclipse is also composed of an-auto completion system for functions, real-time syntax-errors detection as well as an included debug tool to execute programs step by step.

4.2.1.4 Java as programming language

Java is a high level programming language originally developed by James Gosling at Sun Microsystems and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to bytecode (class file) that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is a general-purpose, concurrent, class-based, object-oriented language that is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere" (WORA), meaning that code that runs on one platform does not need to be recompiled to run on another. Java is currently one of the most popular programming languages in use, particularly for client-server web applications, with a reported 10 million users. [N7]

Android supports Java 1.5 and translates the byte code to its own custom Dalvik byte code optimized for mobile devices. Java provides a very rich API right out of the box and produces the most portable applications since there is a Java VM for almost every platform.

4.2.1.5 Additional APIs and Libraries

Sometimes, the developer has got to be smart enough to not remake complex functions that already exist out there for free. This is why we had to look for some sort of frameworks, APIs

and libraries that would facilitate the programming process for us. Here are the additional tools we integrated in our Android platform:

4.2.1.6 Indoor Maps and Wi-Fi Positioning API

This API was designed by Ericsson Labs to facilitate the integration of indoor location-based services into an android application. It contains three main tools:

- Map Studio: to create custom indoor maps and even design templates for the maps. These Maps and Styles are then stored as XML-files in the Ericsson Labs server.
  A detailed explanation of how we used this tool to generate our shopping mall map can be found in the Annex.
- Android Map API: to easily integrate the Maps into the Android application. It gives the user functions for rendering, interacting with and managing the Map like customized map display, overlaying items and personalizing the tags.
- The Indoor Positioning API: to facilitate determining the user's position using Wi-Fi. Ericsson Lab provides the user with a Wi-Fi Mapping Client, an Android app that the user must install on his smartphone beforehand, to upload the information and the position of the Wi-Fi access points. To locate a user, the application will scan the surrounding Wi-Fi access points and send all of that data to Ericsson Labs service via the API and get an Indoor Position back.

4.2.1.7 Barcode Decoding using ZXing Library

There are several libraries available to decode barcode symbologies. A select few of these libraries are open source. One prominent open source library is the ZXing project. It has the capability to read not just 1D barcodes but also 2D barcodes and is based on the Android platform. We integrated this library to facilitate the barcode scanning and decoding process.

4.2.2 The Server-Side (Back-End): Web Development

4.2.2.1 WAMP server 2.1 as Environment

WAMP is acronym for Windows, Apache, MySQL and PHP, Perl, Python.

It's a one-click installer collection of web development tools and soft wares, which provides an environment for developing web pages and applications. It contains Apache Web Server, MySQL Database Management System & PHP Programming Language.

Since we wanted to install the three main components for web development all in one, we used WAMP Server for its ease of use interface and its open-source availability.

4.2.2.2 PHP 5.3 as a scripting language

Since interacting with our website necessitates a login process along with basic CRUD (create, read, update, delete) operations to be performed on our database, there was no doubt we had to use PHP as powerful server-side scripting language.

PHP is a scripting language designed to fill the gap between SSI (Server Side Includes) and Perl, intended largely for the web environment. PHP has gained quite a following in recent times, and it is one of the forerunners in the Open Source software movement. Its popularity derives from its C-like syntax, and its simplicity.

Basically, PHP allows a static webpage to become dynamic. "PHP" is an acronym that stands for "PHP: Hypertext Preprocessor". The word "Preprocessor" means that PHP makes changes before the HTML page is created. This enables developers to create powerful applications which can publish a blog, remotely control hardware, or run a powerful website such as Wikipedia or Wikibooks. Of course, to accomplish something such as this, we obviously need a database application such as MySQL.

4.2.2.3 MySQL 5.5 as the database

Since we have to store information about the potential shopping mall managers who in turn, will be using our website to store their product information, we have to use a powerful database management system and MySQL seems like the perfect fit, especially when it's already included in the WAMP package.

MySQL is the world's most used relational database management system (RDBMS) that runs as a server providing multi-user access to a number of databases. The SQL phrase stands for Structured Query Language.

MySQL is a popular choice of database for use in web applications, and is a central component of the widely used WAMP open source web application software stack.

It is an open source database management system and is used in some of the most frequently visited websites on the Internet. Many programming languages with language-specific APIs include libraries for accessing MySQL databases. These include MySQL Connector/Net for integration with the JDBC (Java Database connectivity) driver for Java. [N9]

4.2.2.4 APACHE 2.2 as the web server

Now that we have already defined our scripting language as well as our database, we need to look for a web server to host our web pages. Once again, the choice was cast on a WAMP-included component which is the Apache server.

Apache is a freely available Web server that is distributed under an open source license.
According to a Netcraft (www.netcraft.com) Web server survey, 60% of all Web sites on the Internet are using Apache (62% including Apache derivatives), making Apache more widely used than all other Web servers combined.

4.2.3 Web Services deployment:

4.2.3.1 PHP classes as web services and Android as a RESTful Client

To display information from a database to a mobile, one mediator is required.
Web services provide a simple tool to facilitate the communication between heterogeneous applications. Since we need to update and retrieve products information from the database, we had to call web services to easily perform these operations.
We implemented the web services using PHP and the output is in JSON format as it is easier to parse than its counterpart XML.
The implementation of the web services is done in an independent way. Each script was written apart to ensure a specific function without necessarily interacting with the other modules. For a later deployment, all the scripts were then stored in the Wamp/www/ directory.

Having the JSON object available as a data format for web services means that we can create (insert), update, and query any data using the JSON object format.
To consume these web services by the android client, we need to send a request to the desired PHP web service file using a Background Async task thread to connect the android device to the PHP script. The PHP script will fetch data from the database, encode it into the JSON object format and send it back to the device. The application will get the encoded data as a response and will eventually need to parse these results for a convenient display on the android device. This is why this method resembles the REST approach of deploying web services.

4.2.3.2 JSON as exchange format

JSON or JavaScript Object Notation is a lightweight text-based open standard designed for human-readable data interchange. It is derived from the JavaScript scripting language for representing simple data structures and associative arrays, called objects. Despite its relationship to JavaScript, it is language-independent, with parsers available for many languages. The JSON format is often used for serializing and transmitting structured data over a network connection. It is used primarily to transmit data between a server and web application, serving as an alternative to XML.

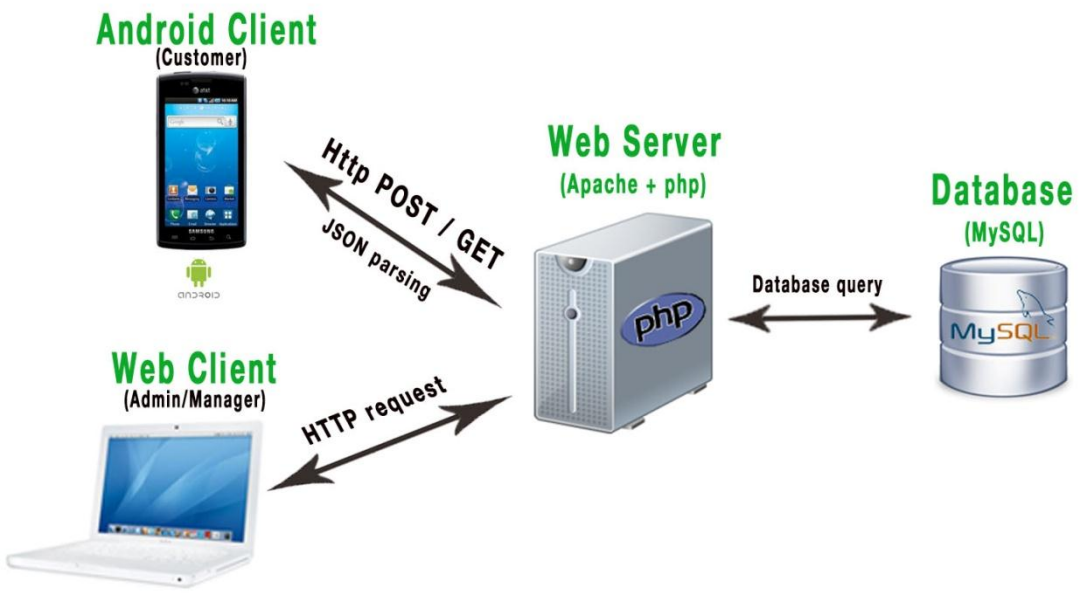


Figure 4.2 Implementation from the clients and server perspective

4.2.3.3 HTTP Network Protocol

As mentioned before, the application is being designed in such a way that it is capable of communicating with outside sources (External Database Server, Ericsson Lab Server) via Internet for obtaining information on products and map viewing. Hence, the application will use standard TCP/IP sockets to communicate data over Hyper Text Transfer Protocol (HTTP) to remote and external servers.

The application's use of HTTP for communication is well justified as the standard for Internet. The protocol provides a simple method to request and send data, which is exactly what we need to accomplish.

A HTTP request is sent from the client to the server, usually requesting a webpage to be delivered. The most used commands in HTTP are GET and POST, which request a file from the server and send information to the server respectively.

4.2.4 Other Softwares used

4.2.4.1 Power AMC 15.1

There are many tools available for drawing UML diagrams, varying from professional ones like Rational Rose to more traditional open-source tools like ArgoUML. For its simplicity and its ease of use, we chose to work with Power AMC in order to generate all the diagrams that we presented in the design chapter.

4.2.4.2 Photoshop CS5

Photoshop is considered to be among the most powerful design tools. Since we had to conceive an intuitive ergonomic user interface, and to come up with a cool map for our virtual shopping mall we used Photoshop to generate all the necessary images.

4.2.4.3 Dreamweaver 8

For the development of our server side web application we have used Macromedia Dreamweaver 8, on which we have written our PHP, HTML and CSS code.
It provides an intuitive interface allowing us to graphically view our instant updates regarding the design of the pages without having to reload them on our local server each time.

4.3 Final results presentation

In this part, we will present the final results of our implementation. You will be able to see what our application interface looks like from each user perspective through a collection of screen shots depicting each possible use case.

4.3.1 Android client perspective

We tried to do our best to design a genuinely intuitive and interactive user interface that meets up with the ergonomic guidelines we had previously set up in our non-functional requirements. We can't deny that our Photoshop skills and the XML magic tricks we learned through tutorials were of very precious help.

4.3.1.1 Main Menu

Any android application consists of a collection of pages that could be displayed to the user. Such a page is called and “activity”. The user navigates from one activity to the other through manual input or click events. Below is the main “activity” in the Mobile Shopping Assistant.

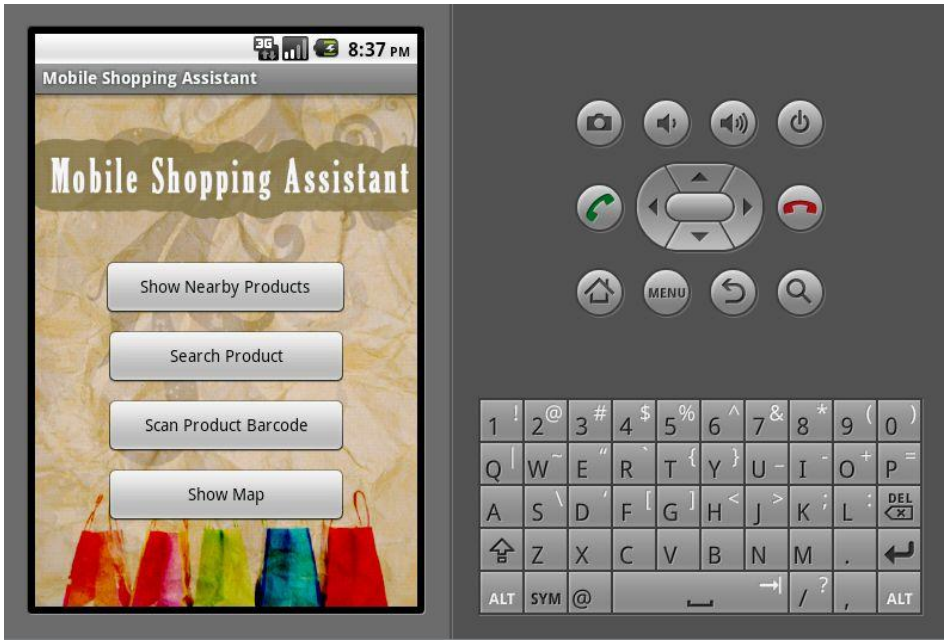


Figure 4.3 Main Menu Android Interface

Even though the customer doesn’t need to log on to an account to benefit from our features, once he chooses an option from the menu, the WI-FI manager on his phone will be activated, thus allowing the communication between the device and the server to take place. The device is naturally being recognized from its unique MAC-address on the Wi-Fi network.

4.3.1.2 Show Nearby Products

If the user chooses this option, as we mentioned above, the Wi-Fi will be automatically activated and the list of products that will pop up will be dynamically updated according to the user’s position. He will be associated with the closest Wi-Fi access point having the strongest strength signal within his range.

4.3.1.2.1 Loading Products' List



Figure 4.4 Loading Products list Android interface

While the calculations associated with the Wi-Fi positioning algorithm are being made in preparation of the database query through the consumption of the respective web services, the list of products is being lazy loaded from the database. Only products that have the corresponding aisle tag as an attribute will appear.

4.3.1.2.2 Viewing Products List



Figure 4.5 Viewing products list Android Interface

In this case, the customer appears to be standing in the men's wear area. This particular aisle matches the reference tag returned by the Wi-Fi positioning algorithm.

This is why we can only view the items stored in that particular department.

The user can however refresh the page and the list will dynamically be updated according to the customer's position.

4.3.1.2.3 Viewing a particular product's information from the list

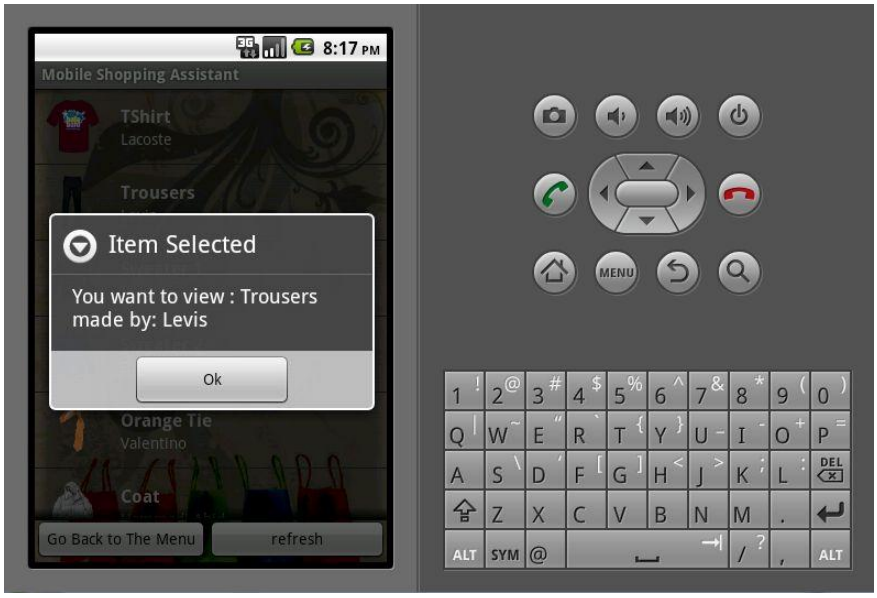


Figure 4.6 Look Up Product Info Android Interface

The user has earlier chosen to click on the trousers item. He gets a notification to proceed with the product info lookup.

4.3.1.3 Search Product

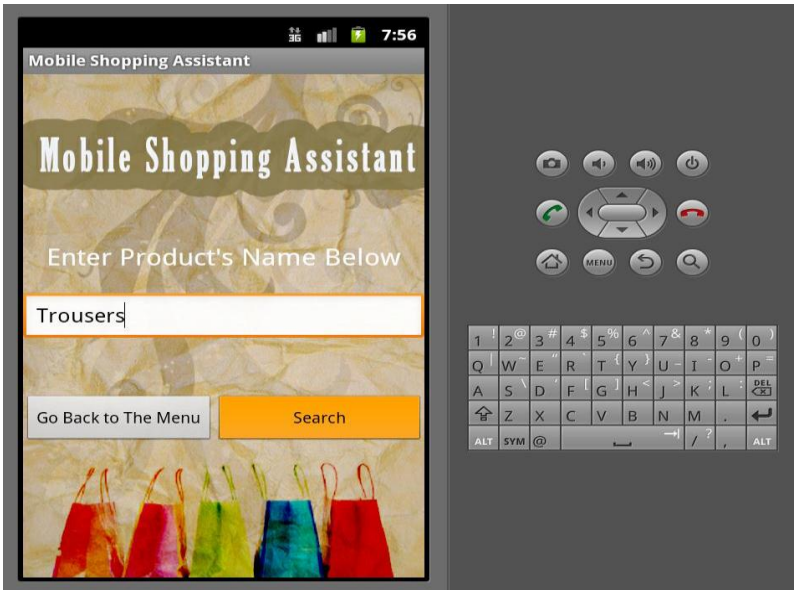
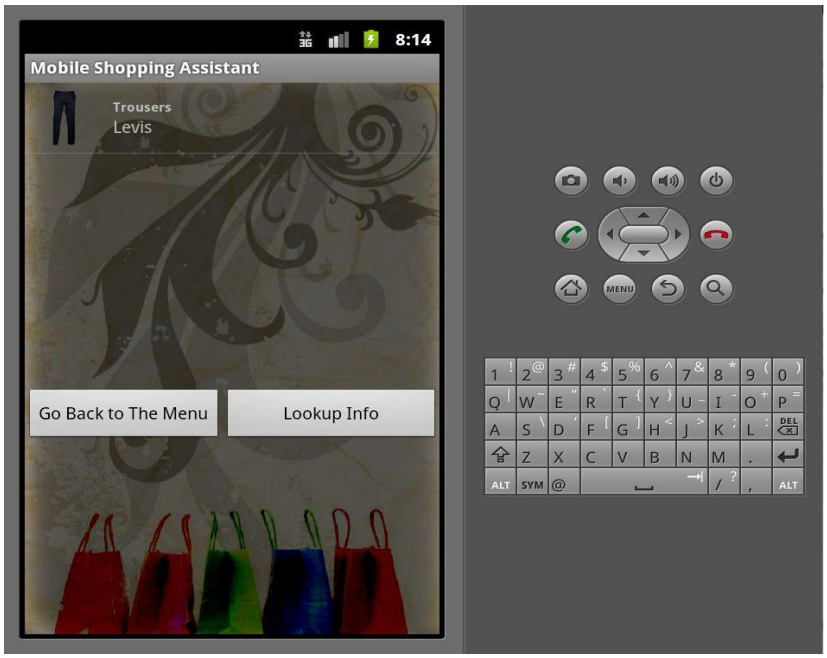


Figure 4.7 Manual Product Search Android Interface

This screen is for manual search of the product. If the customer desires to look for a particular product, he has to manually input its name then hit the search button.

4.3.1.3.1 Product Found



This screen appears once a match has been found. The customer will now be able to view the item he was looking for. He can choose between looking up the product info and going back to the main menu.

4.3.1.3.2 No matches found

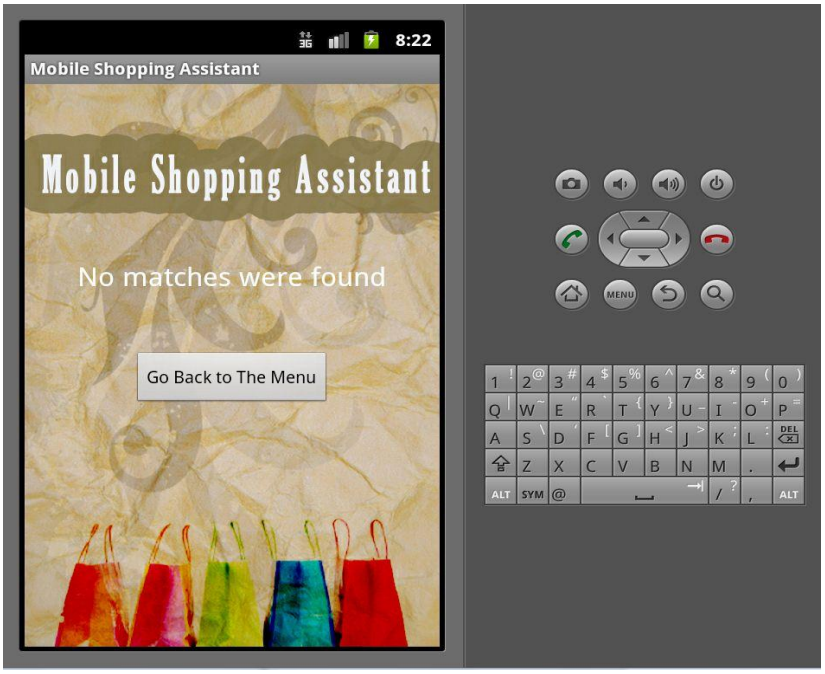


Figure 4.8 Unfound data Android Interface

This is the standard screen that the user might get if his request has not been fulfilled.

4.3.1.4 Scan Product Barcode

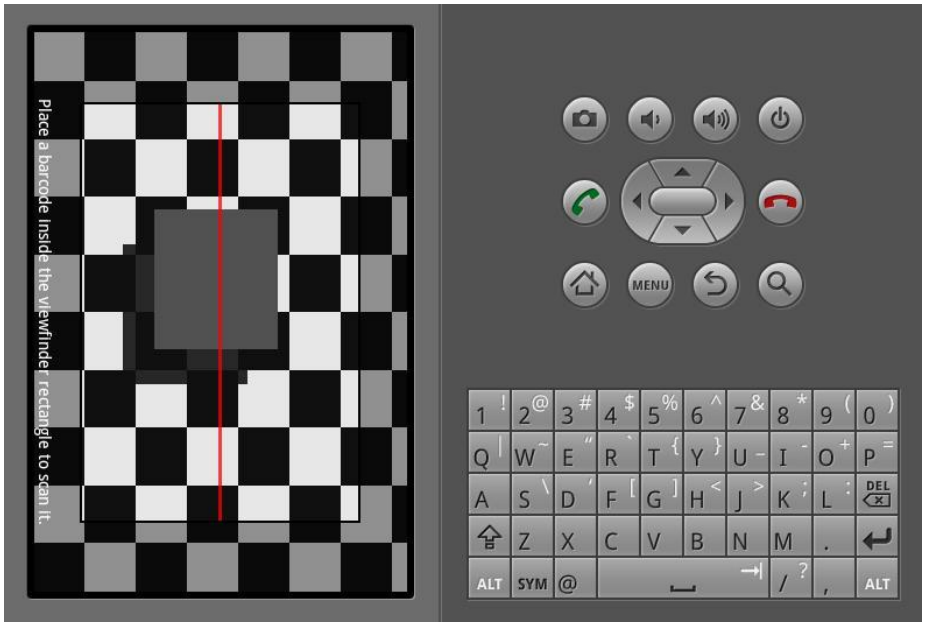


Figure 4.9 Scan product barcode Android Interface

Since we used to the emulator to capture all the screenshots, we unfortunately couldn't activate the camera function as we need a real phone to be able to do that. This is however the screen we get if the user wants to scan the barcode of a particular product.

4.3.1.5 Rate a product



Figure 4.10 Rate product Android Interface

This is the same screen that pops up once the user has done either of the following choices: search a particular product and lookup its info, select a product from the list of nearby products to view it, or successfully barcode scanning an item.

All the information regarding the product is displayed in order. At the very end, the customer can rate the product by highlighting a selective set of stars, from 1 to 6.

The rating count will then be incremented accordingly.

4.3.1.6 Show Map

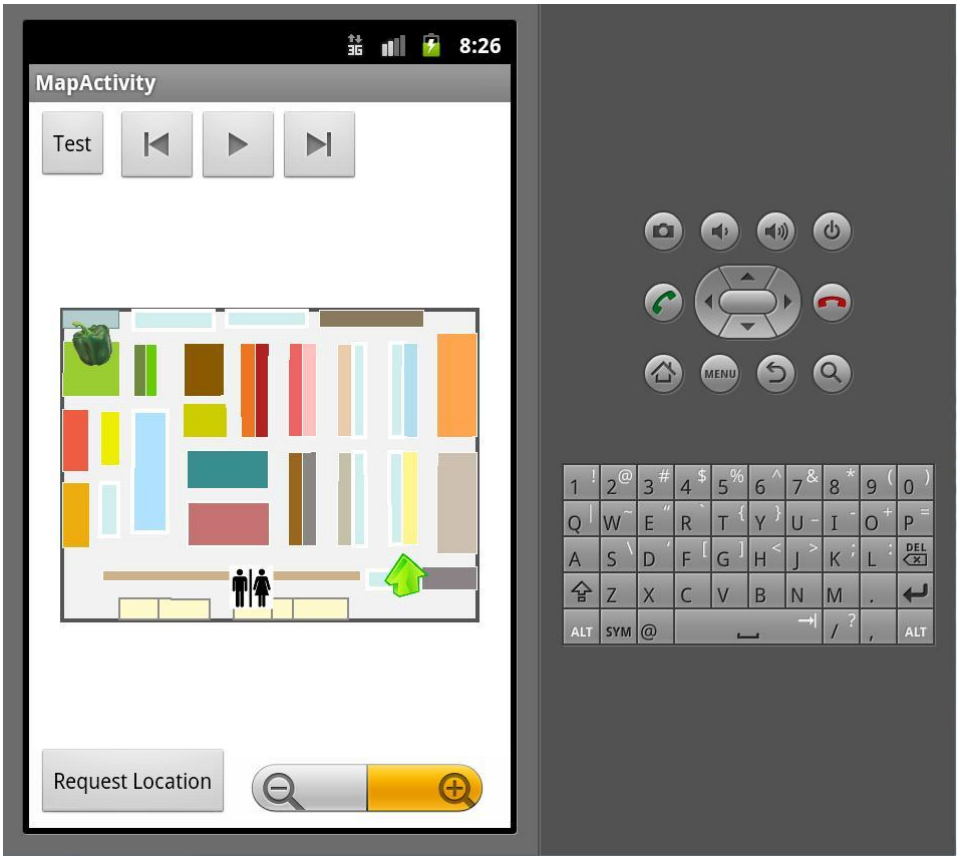


Figure 4.11 Show Map Android Interface

This is the part where we integrated the Indoor Maps and Positioning API. The user's position is updated according to the Wi-Fi positioning algorithm we mentioned earlier. This feature requires however, that the Wi-Fi access points of the entire shopping mall have already been mapped by the application developer to make it easier for a common user to be able to locate themselves around the mall without having to do the mapping themselves. We didn't get enough time to put all the icons representing each aisle/area, or additional reference points. Here you can see that the entrance is where the green arrow is, the restroom and vegetables area are also appearing with their respective icons.

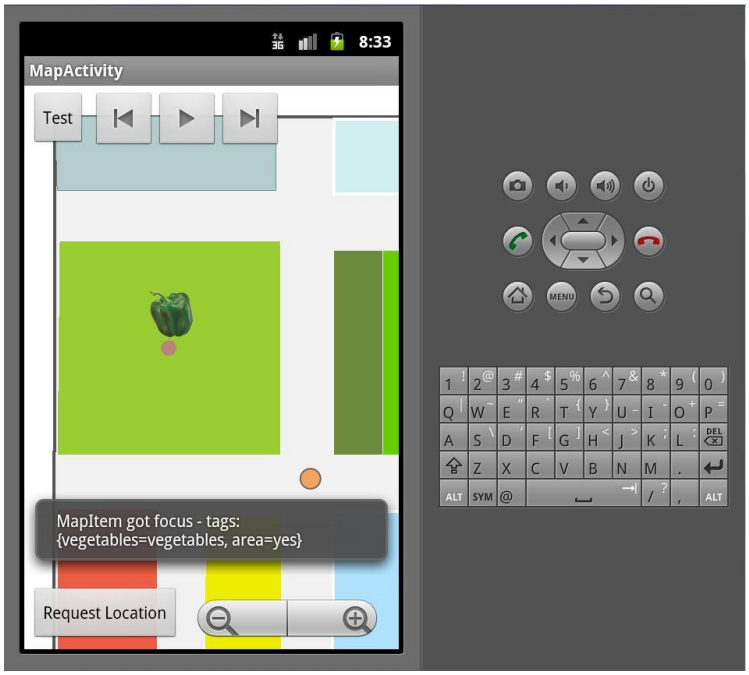


Figure 4.12 Zoomed in Map Area tag Android interface

If the user wants to zoom in on a particular area, he will be able to view the orange dot which points to a certain position within the shopping mall and another violet dot if he points on a selected area. In this case the user is within short distance from the vegetables area.

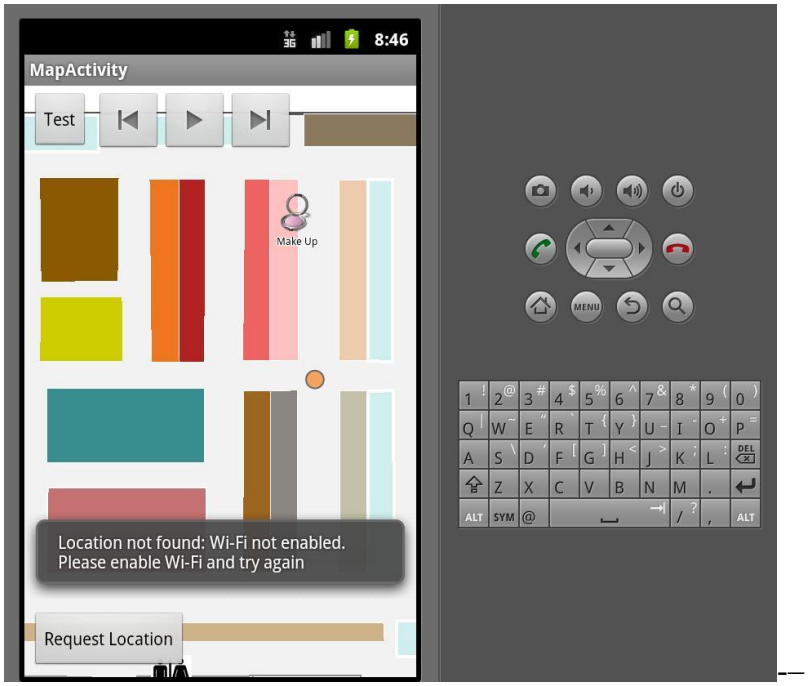


Figure 4.13 Wi-Fi not enabled Android Interface

This screen appears if the user clicks on the Request Location button. Since the Emulator doesn't support the Wi-Fi feature, these are only static theoretical results.

4.3.2 Web client perspective

4.3.2.1 MSA web application: the Welcome & Login Pages

For the manager and the administrator of the Mobile Shopping Assistant, we have conceived a web application through which both can manage the data stored in the database.

First of all, the "welcome" screen appears on the browser page:



Figure 4.14 Screenshot of the web application welcome page

The concerned person has then to log in whether it is a manager or the database administrator. He can click on the green link "click" or on the "Login" tab on the top. The following screen appears:

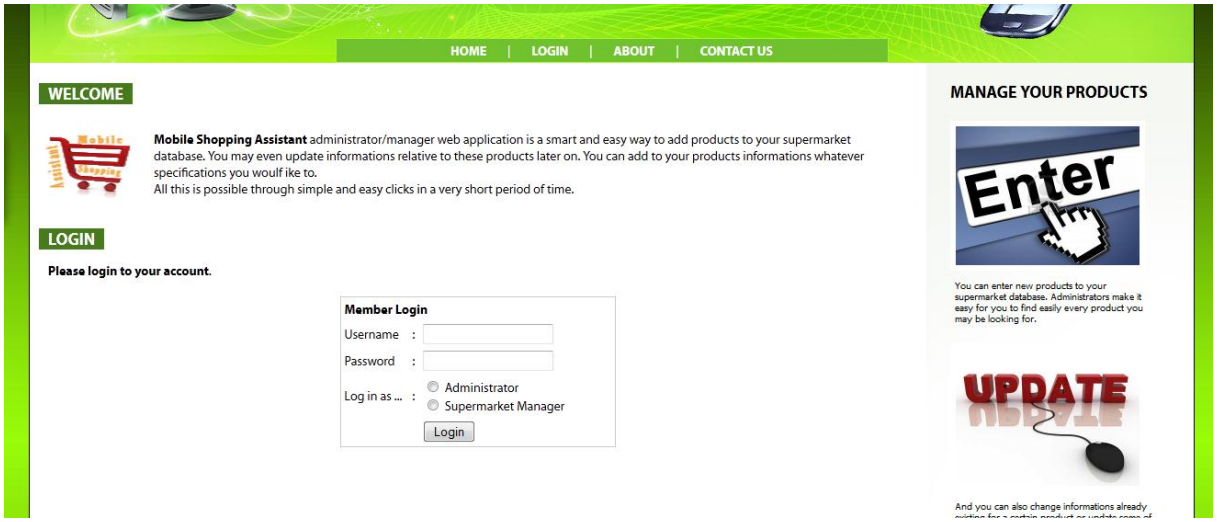


Figure 4.15 Screenshot of the web application login page

The user is asked to enter his username, his password and the way he wants to connect, whether as an administrator or as a supermarket manager. A login request as an administrator from a supermarket manager is treated as an error. Below is an example:

```
INSERT INTO `managers` (`id`, `username`, `email`, `password`, `account`) VALUES
(2, 'Caty', 'caty@google.com', 'caty', 'manager');
```

Figure 4.16 SQL code of the user 'Caty' insertion into the database

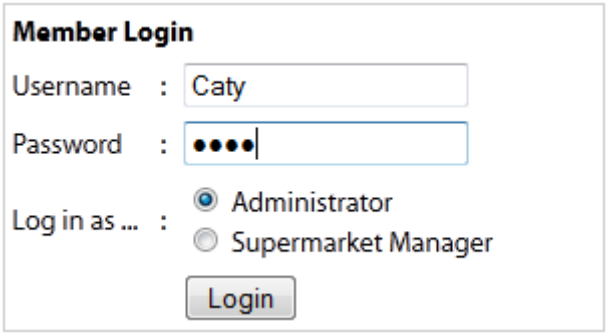


Figure 4.17 Example of false log in mode

Caty, who is a Supermarket Manager, tries to log in as an administrator. Her browser displays the following error message "*Wrong Username or Password*". It's also the same case of the database administrator trying to log in as a supermarket manager.

4.3.2.2 Log in as the database administrator

There is one unique database administrator. He can add, delete or view products from the database, update information related to these products and add, delete or view managers' profiles.

When logged in as an Administrator, the following profile shows up on his web browser:

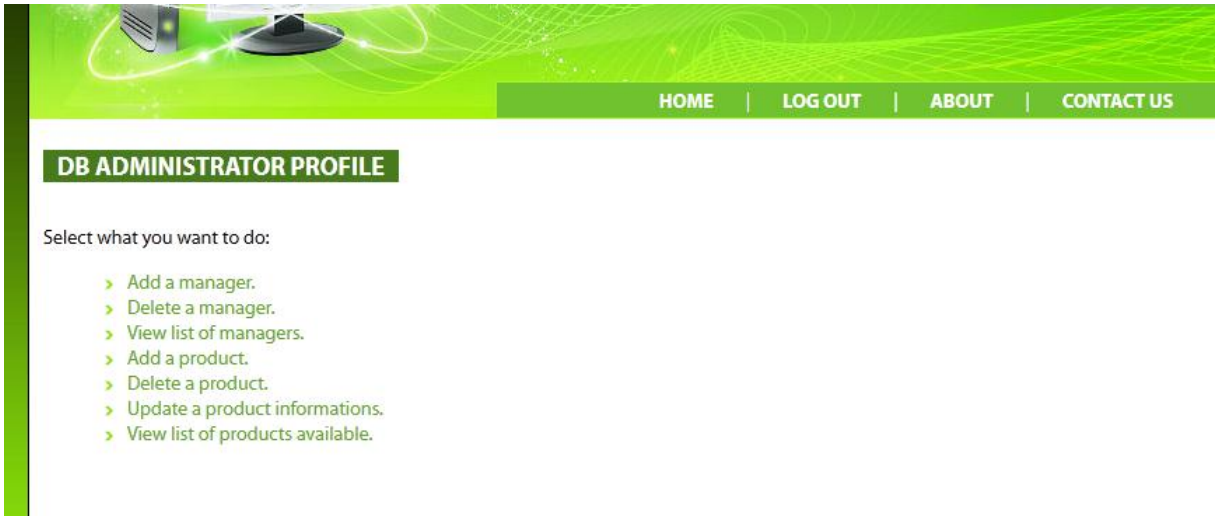


Figure 4.18 Screenshot of the database administrator profile

Below are the different interfaces of the available options.

4.3.2.2.1 Add a manager

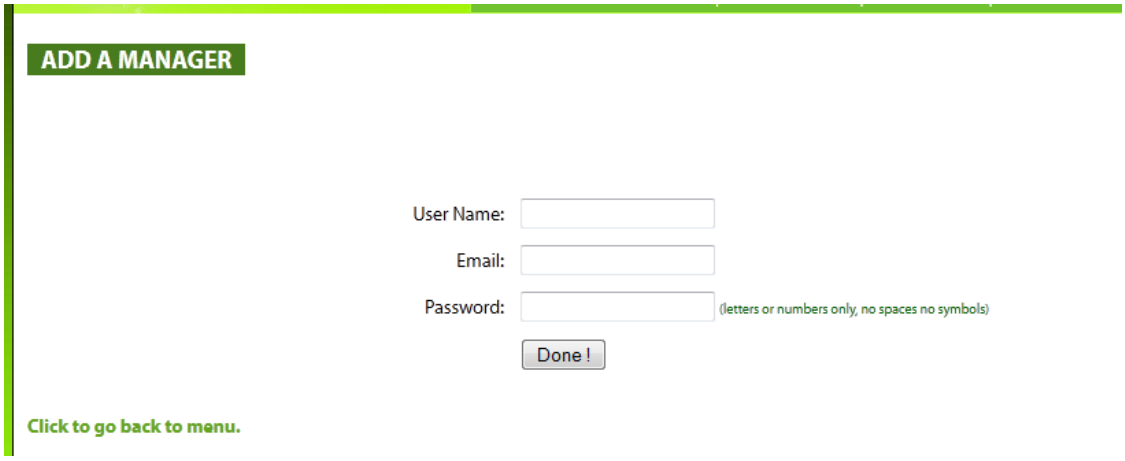


Figure 4.19 Screenshot: Add a manager

The preview above includes a 3-field form. To add a new manager to the database, the administrator enters the manager's user name, his email and the password he assigns him.

4.3.2.2.2 Delete a manager

The next screenshot is about deleting a manager who's already registered in our database. The administrator must just select the manager he would like to delete.

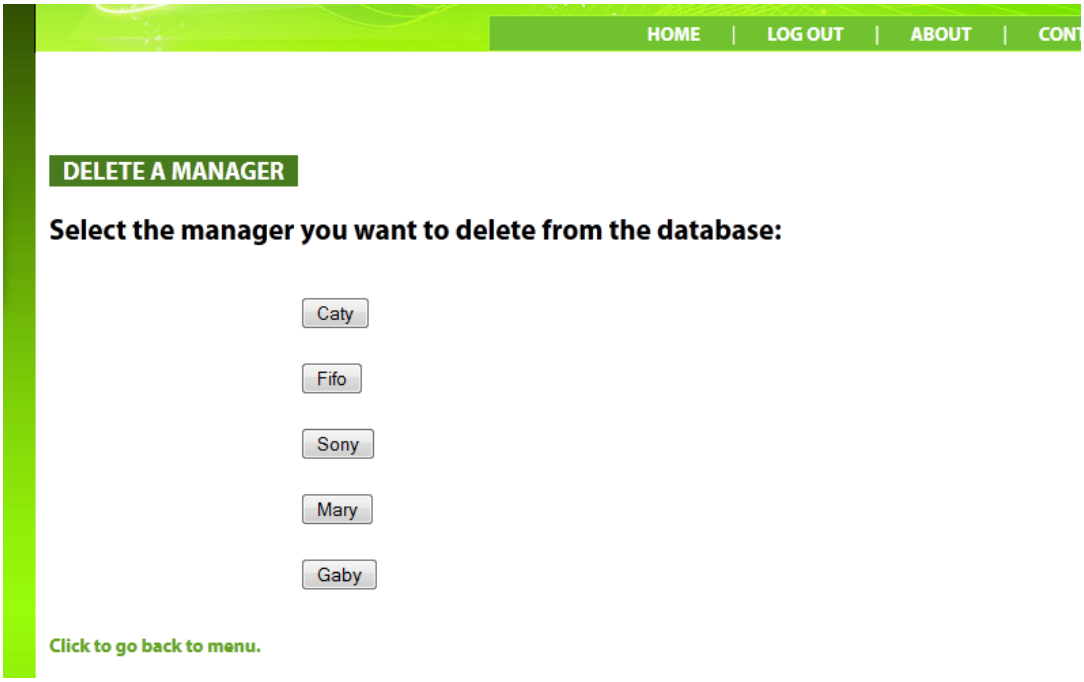


Figure 4.20 Screenshot: Delete a manager

4.3.2.2.3 View managers

The screenshot below shows a table including all the managers existing in the database and the information stored for each one of them.

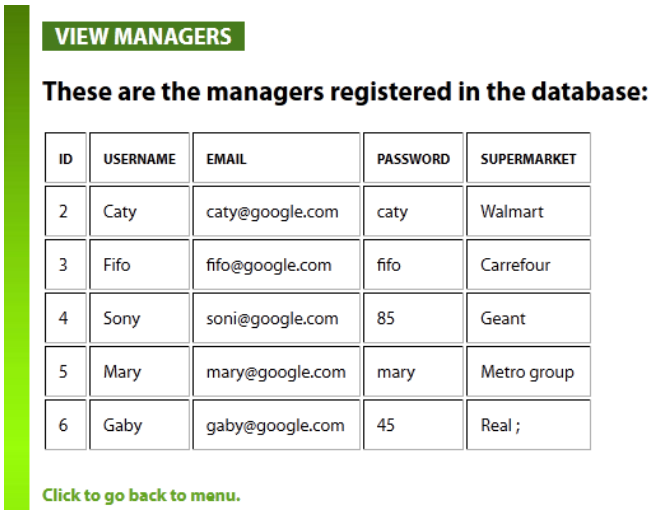


Figure 4.21 Screenshot: View managers

4.3.2.2.4 Add a product

The next interface is about adding a new product to the database. The administrator has to fill in a form with the product information needed such as the manufacturer, the price, etc. Some fields are optional such as assigning an image or putting a discount.

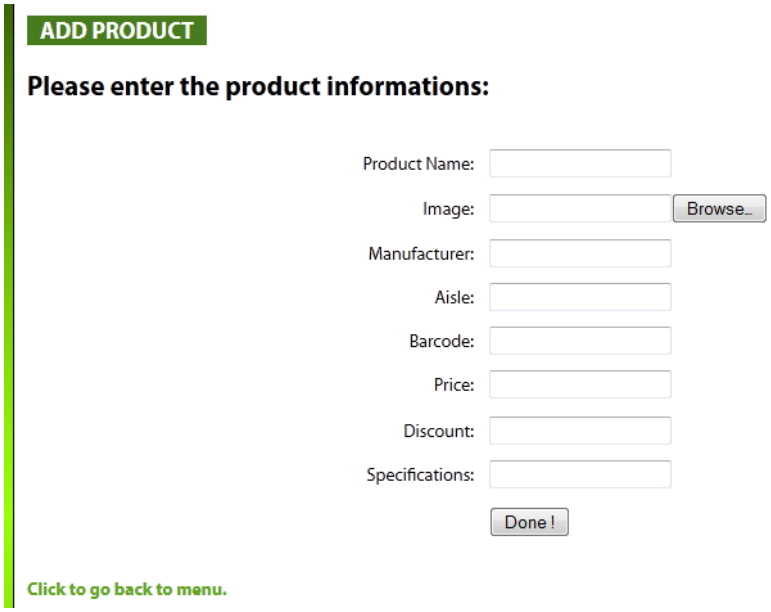


Figure 4.22 Screenshot: Add a product

4.3.2.2.5 Delete a product

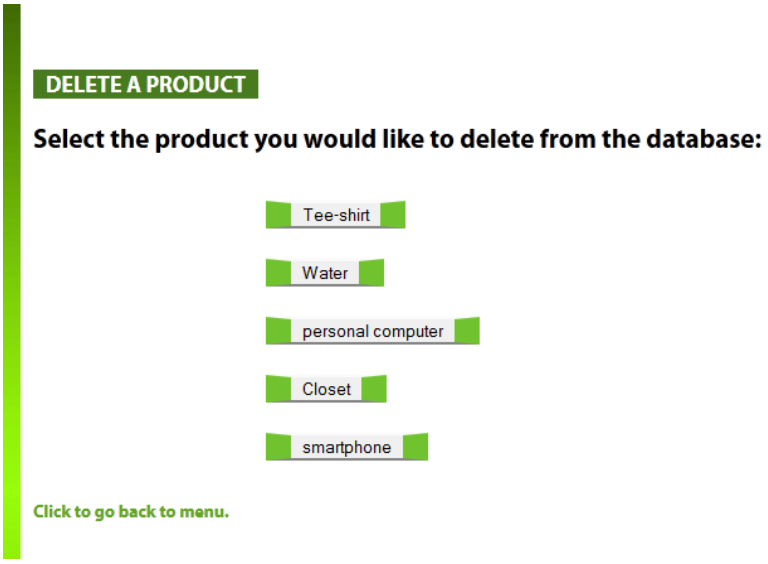


Figure 4.23 Screenshot: Delete a product

Above we have a preview of the "delete a product" option. The products available in the database are listed on the screen, and the administrator clicks on the one he would like to delete.

4.3.2.2.6 View products

This option allows the administrator to view a table of all the products he stored in the database with the information related to each one of them. The next figure shows a preview of it.

VIEW PRODUCTS

Here are the products registered in the database:

ID	PRODUCT	MANUFACTURER	AISLE TAG	BARCODE	PRICE (DT)	DISCOUNT (%)	SPECIFICATIONS	RATING	IMAGE
1	Tee-shirt	Lacoste	Men	6191577600015	20		100% Cotton	12	
2	Water	Sabrine	bavarages	6195898540065	0.6		Mineral Water.	65	
3	personal computer	VAIO	hi-tech	9856325639856	1500	10	i5 processor 4GB RAM 500GB Hard Drive Sata 2GB Graphic NVIDIa Bluetooth, wifi, windows 7 premium	3	
4	Closet	IKEA	Fourniture	454841289	25	0		0	
5	smartphone	Motorola	Hi-tech	46878541566	235.95	20	Motorola	0	

Click to go back to menu.

Figure 4.24 Screenshot: View products

Thanks to a barcode generator PHP code found on the internet [http://barcode-coder.com/], the administrator can view the barcode of the product just by clicking on it. Here's for example the barcode generated for the first product, the Lacoste Tee-Shirt, which serial code is 6191577600015:

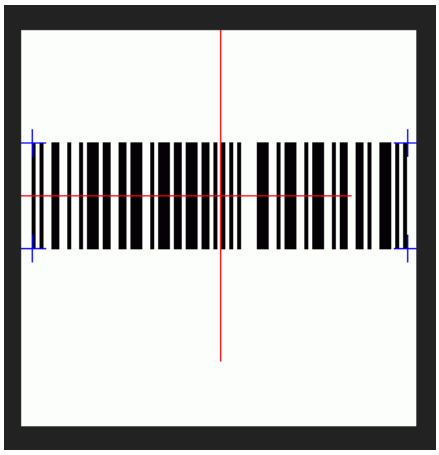


Figure 4.25 Barcode relative to 6191577600015

4.3.2.2.7 Update a product

The administrator is also able to update a product. He selects from the products listed, the one he wishes to update and a form similar to the "add a product" one shows up. Below is a screenshot of this interface.

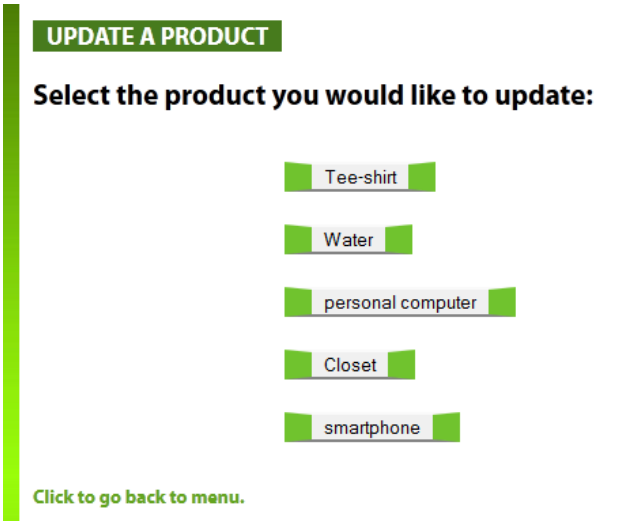


Figure 4.26 Screenshot: Update a product

Every time the database administrator makes a change on the database, if the change has been processed successfully to the database, the following is shown:



Figure 4.27 Screenshot: Database up to date

After updating both of his databases, managers and products, the administrator can log out by clicking on the 'Log out' tab on the top.



Figure 4.28 Screenshot: Web application top banner

4.3.2.3 Log in as a supermarket manager

A supermarket manager is a user who was registered, in advance, by the database administrator. A supermarket manager is defined by a username, an email and a password.

When logged in, the following menu appears on his browser:

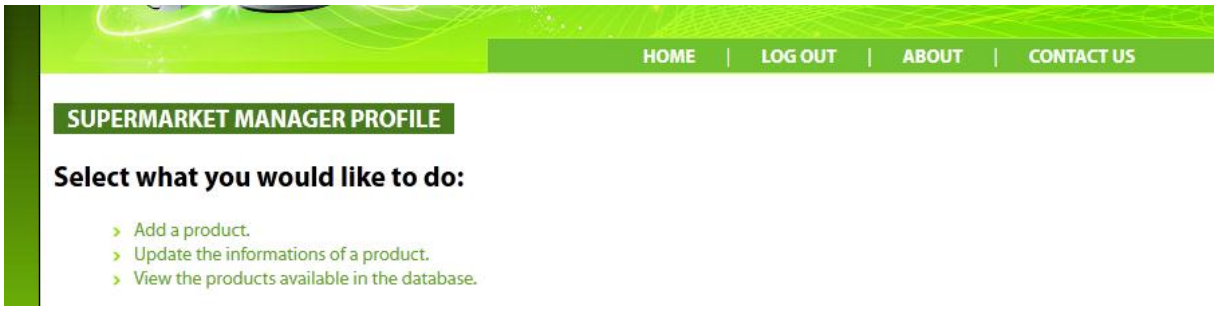


Figure 4.29 Screenshot: Login

A supermarket manager is able to add a product, update an existing product's information or view which are the products already registered into the database.

Below, we show the interfaces of the different options available for a supermarket manager.

4.3.2.3.1 Add a product

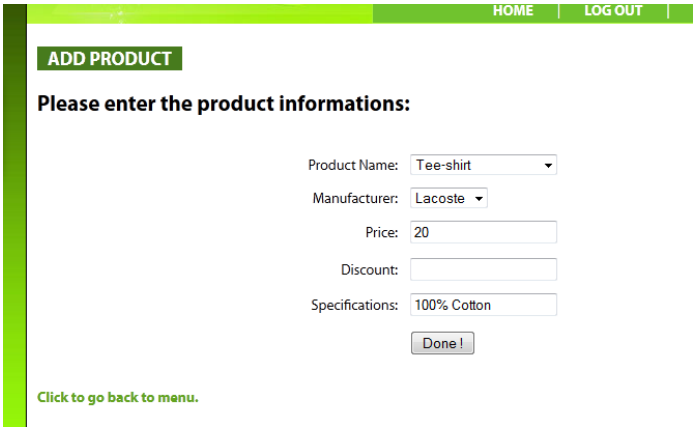


Figure 4.30 Screenshot: Add a product

A drop-down menu appears showing the products already existing in the administration database. The supermarket manager chooses the products he wants to add to his supermarket database. A list of manufacturers of this product shows up. The relative image, aisle and barcode are automatically associated without the manager's intervention. The supermarket manager can choose another price for which he would like to sell this product and the discount he may put. He can also add some specifications.

When clicking on "Done !", the information added will be stored into the database.

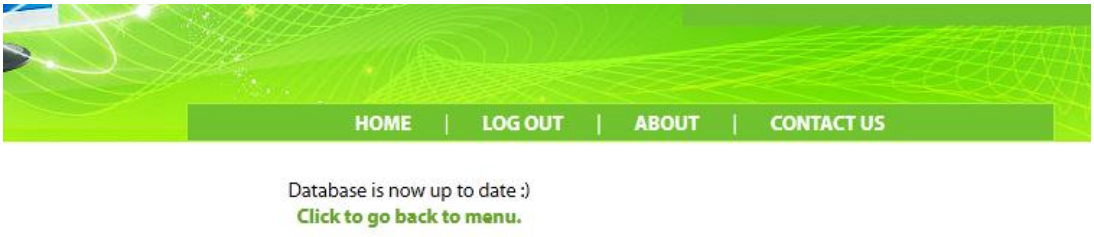


Figure 4.31 Screenshot: Database up to date

4.3.2.3.2 Update a product

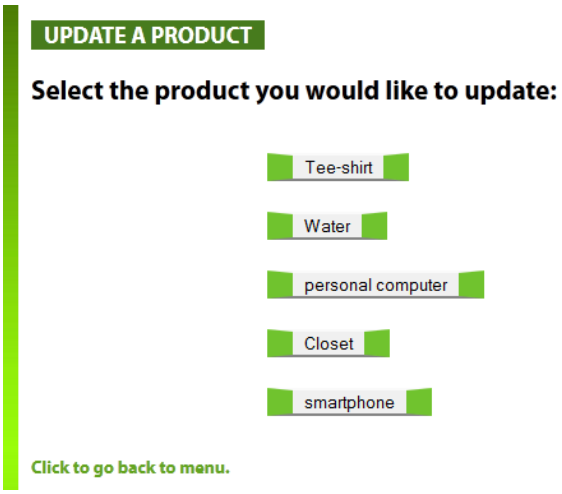
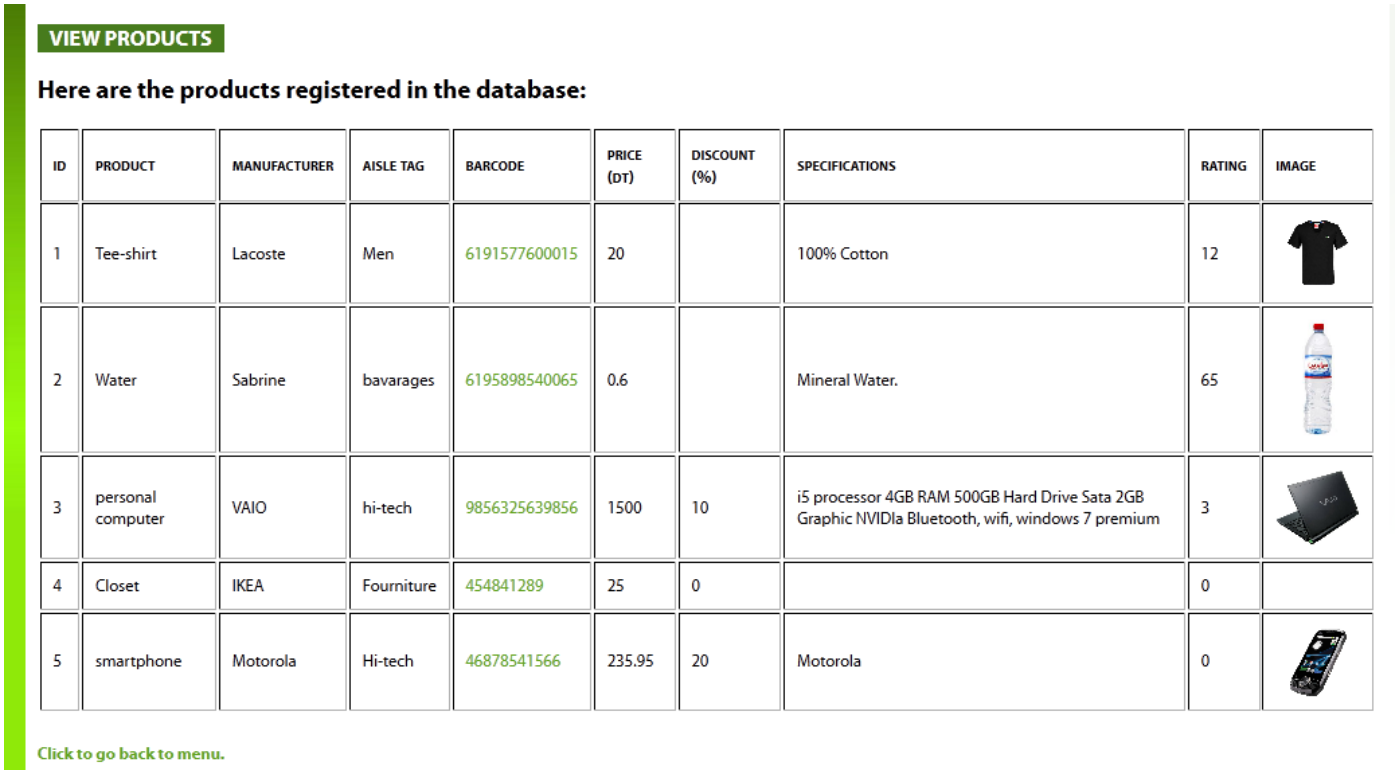


Figure 4.32 Screenshot: Update a product

The screenshot here above shows the update product menu. The manager has to click on the article he wants to update.

4.3.2.3.3 View products

The supermarket manager may also view the products available in the database. A screen similar to the administrator view product screen show up.



ID	PRODUCT	MANUFACTURER	AISLE TAG	BARCODE	PRICE (DT)	DISCOUNT (%)	SPECIFICATIONS	RATING	IMAGE
1	Tee-shirt	Lacoste	Men	6191577600015	20		100% Cotton	12	
2	Water	Sabrine	bavarages	6195898540065	0.6		Mineral Water.	65	
3	personal computer	VAIO	hi-tech	9856325639856	1500	10	i5 processor 4GB RAM 500GB Hard Drive Sata 2GB Graphic NVIDIa Bluetooth, wifi, windows 7 premium	3	
4	Closet	IKEA	Fourniture	454841289	25	0		0	
5	smartphone	Motorola	Hi-tech	46878541566	235.95	20	Motorola	0	

Figure 4.33 Screenshot: View products

The same option to view the barcode image is also available for the supermarket manager.

4.4 Project Timeline

The development of this project has started from February 6th and is still going on till May 22nd 2012. First of all we began an extensive research of the existing solutions, otherwise known as the state of the art. During this phase we started reading a lot of documentation to get in the spirit of this project and especially to be able to specify the requirements. After having fully understood and expressed the desired work, we were able to fully specify both the functional and non-functional requirements. We then had the task of elaborating the different diagrams in the design phase. For this stage, we had to choose the according architecture that our application is going to embed along with the database schemas we are going to work with. Once we had our design elements in place, we started training on the different technologies we are going to use in order to implement our application. In our search for organized time management and an effective planning for our project development, we came up with this project flow chronogram detailing the different tasks that our project contains.

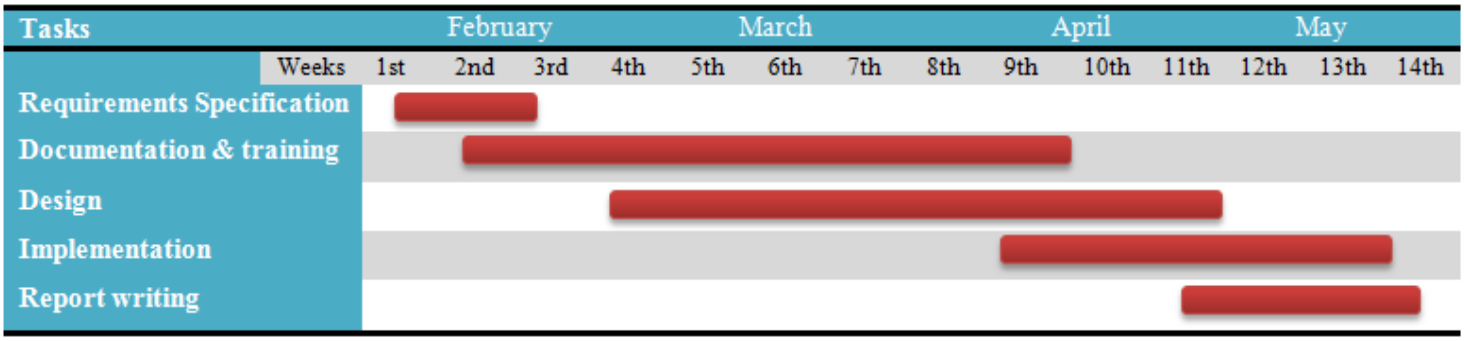


Figure 4.34 Project Timeline

4.5 Problems faced during the project development

From a technical point of view, the first major problem we faced since the beginning of the project is in regards to the process of locating a client inside the shopping mall. Since GPS works very poorly indoors and gives minimum accuracy we had to choose between Wi-Fi and 3G connection. We then had to consider that not all clients possess 3G internet connection as opposed to Wi-Fi network being usually in almost all major shopping malls. The second problem that was raised at that moment was how to make sure each aisle had a particular Wi-Fi access point and a rather stable signal strength. Here we just had to assume that the manager already knows the BSSID, RSSI and the coordinates of each Wi-Fi access point. Up to this moment, we were still stuck in how to get all these information into our database without concrete mapping of the Wi-Fi access points' structure. At the very end, we found a pseudo-solution to this problem by working with the Ericsson Labs Indoors Wi-Fi Positioning

API that provided us with a Wi-Fi Mapping Client, which is basically an app to be installed in the Androphone to collect the specific data regarding the access points positions and do the corresponding mapping accordingly.

The major problem here is we didn't even have a real smartphone on which we could install this app to do the mapping for us. All we worked with was the Eclipse Emulator which didn't match all the requirements of a real smartphone. It doesn't simulate Wi-Fi nor activates the Camera. This means that, unless we get hold of a real device we can't really test the functionalities of our application, especially the barcode scanning and Wi-Fi positioning.

Another issue was brought up when we had to identify the shopping mall items. The two main data collection technologies we considered were RFID and barcodes. Although RFID might have more advantages than barcodes, like a wider read range, not requiring a line of site, being automated as opposed to the labor intensive barcode scanning approach (which obviously needs the human to operate), RFID will not completely replace barcode technology. Barcodes offer some advantages over RFID, most notably their low cost. We chose to deal with barcodes instead of RFID tags as they are more common and their decoding is easier. This choice fortunately worked to our side, when we have eventually found out about the existence of external libraries that facilitate the barcode scanning and decoding process like the open source library ZXing which we integrated into our application.

From a software development point of view, we had trouble identifying which approach of web services are we going to deploy. We had to go through a long process of documentation to choose between SOAP and REST. Finally we ended up choosing PHP scripts which resemble the REST approach and are a lot easier to work with than both the above styles.
Another problem was the data persistence layer, as our application still lacks this aspect. We couldn't decide whether it was necessary to apply Hibernate or JDBC as both are very hard to implement. Since we connected to the database using the web services and JSON to parse the results for us, we didn't have to add another layer for persistence.

4.6 Possible Future Improvements

Since we didn't have enough time to think of more interesting functions that our app could provide, we hope that if the project carries through we can possibly deploy it on a larger scale, thus including even more profitable features like displaying user reviews of the products and

not just a rating, create personalized customer profiles and user accounts to authenticate the client while allowing him to make real business transactions, gain special offers and coupons, add shopping lists with a virtual cart or even guide the client to collect ingredients for a customized recipe.

On the implementation side, we can think of more ways to improve the data exchange, and especially add a persistence layer to enhance the security.

4.7 Conclusion

We hope that through this chapter the implementation was clear enough as we presented both the hardware and software environment along with a justified choice for the technologies we opted for. We tried to put some efforts to make the navigation through the different interfaces go as smoothly as possible. We still need to point out that despite all the technical problems we faced, what we just presented was a mere prototype that could hopefully be subject to many possible improvements.

GENERAL CONCLUSION

Mobile Marketing has become the new trend in the world of marketing. Even if its impact is still not widely perceivable for the Tunisian community as the percentage of smartphone-holders here still have not reached the same percentage it has in other countries, we still think that the figures are going to be on the rise soon enough. If we make the buzz for this freshly out-of-the box concept of offering indoors location-based services for the customers and making the core stakeholders that are affected by this system even more involved in promoting it, things will start to pick up in a significant way, on many levels.

As an effort to put these theories into practice, our work focused on creating an application that embodies the concepts of mobile shopping assistance while integrating indoor-positioning techniques inside the shopping malls to offer location-based services. The application was destined to provide guidance and support to the customer while he is shopping.

To be able to do this, we elaborated a plan for our work flow. The first step was to start collecting data regarding the different theoretical aspects related to our project. On a second note, we started getting documentation on the Android platform and taking a look at the existing M-Marketing solutions in the market as well as indoor-positioning techniques. During the next phase, we started specifying the requirements of our application through elaborating the use cases and the detailed sequence diagrams. The fourth part was to categorize our application under a main design architecture pattern, dividing it into layers accordingly and come up with a database schema that fits our solution's requirements. Now that we had the design model into place, all what was left was to start coding and implementing our solution according to the guidelines that we have set.

This project has not only offered us a genuine opportunity to consolidate our knowledge in computer-science and software engineering like Java programming or Data definition and manipulation language, but also introduced us to the Android platform and the development of web services applications and Service-Oriented-Architecture.

Besides the newly acquired knowledge, this work has been of great help to us regarding our perception of the human relationships. Even though we lacked a third partner, we still felt a deep connection to the value of team work as we learned how to collaborate and share this enriching experience as well as improve our time management skills. To illustrate our devotion to this cause we even put technology to our service as we used modern ways of communication to sync our data through Dropbox and share our links on Pearltree, not to mention the endless conversations over Skype.

Even though at different stages of our development we hit concrete walls, and had to rethink our strategies, we ended up being fairly satisfied with the output as it was an attempt
To conclude, and in search of perfection, we're hoping that this work can be subject to many upgrades in the future order to optimize its functionalities both on the quantity and quality levels. We could for instance, enhance the security of the application by encrypting the JSON objects, and adding a more powerful persistence layer. We could also think of ways to integrate other services like fidelity-points to earn and retain the customers' loyalty as they are the major players in our stakeholding system.

BIBLIOGRAPHY

Pascal ROQUES, *Les cahiers du programmeur, UML2*, EYROLLES (4e édition)

Reto MEIER, *Développement d'applications professionnelles avec Android 2*, Pearson France 2010

NETOGRAPHY

[N1] http://searchnetworking.techtarget.com/definition/location-based-service-LBS seen on 05/02/2012

[N2] http://www.powermarketing.net/marketing/mobile/

[N3] http://www.mobilemarketingmagazine.co.uk seen on 05/05/2012

[N4] http://www.webopedia.com/TERM/W/Web_Services.html seen on 05/02/2012

[N5] https://labs.ericsson.com/apis/indoor-maps-and-positioning/ seen on 15/04/2012

[N6] http://www.java-samples.com/showtutorial.php?tutorialid=813 seen on 22/04/2012

[N7] http://developer.android.com/guide/basics/what-is-android.html seen on 09/01/2012

[N8] http://en.wikipedia.org/wiki/Java_(programming_language)

[N9] http://en.wikipedia.org/wiki/MySQL seen on 15/04/2012

ANNEX A INDOOR POSITIONING TECHNIQUES

In order to proceed with our development we had to check out the different techniques that already exist. Below is a list of the more common ways we had to consider:



Figure A.1 GPS approach for indoor-positioning

GPS: One possibility is to install transmitters in the supermarket to reproduce GPS signals. GPS based systems can achieve sufficient accuracy, but are unreliable indoors due to signal interference caused by walls, floors, furniture, and other objects.



Figure A.2 RFID tagging technique

RFID tagging: RFID tagging is an ID system that uses small radio frequency identification devices for identification and tracking purposes. Increasingly, RFID tagging is used in supply chain management as an alternative to bar code technology. Unlike a bar code, the tag does not need to be within line of sight of the reader and may be embedded in the tracked object.

Infra-red: Infrared (IR) wireless networking was a pioneer technology in the field of indoor positioning. The primary challenge is the limited range of an IR network. Also, Infrared does not have any method for providing data networking services. An early implementation of an IR technique is the Active Badge System. This is a remote positioning system in which the location of a person is determined from the unique IR signal emitted every ten seconds by a badge they are wearing. The signals are captured by sensors placed at various locations inside a building and relay information to a central location manager system. The accuracy achieved from this system is fairly high in indoor environments. However, the system suffers from several limitations such as the sensor installation cost due to the limited range of IR, maintenance cost, and the receiver's sensitivity to sunlight, which often occurs in rooms with windows.



Figure A.3 Indoor positioning using 3G mobile telecommunications

3G internet connection: 3G or 3rd generation mobile telecommunications is a generation of standards for mobile phones and mobile telecommunication services fulfilling the International Mobile Telecommunications-2000 (IMT-2000) specifications by the International Telecommunication Union.

While this technique might insure rapidity in the retrieval of information, we can't guarantee that every Smartphone user will be interested in turning on his internet connection from the moment he steps into the supermarket. We don't know for sure if the customer does actually have an internet 3G subscription to begin with.



Figure A.4 Using Bluetooth as an Indoor-Positioning technique

Bluetooth: Bluetooth is a wireless communication method used by two devices over short distances. Bluetooth is the IEEE 802.15 standard and is similar to Wi-Fi. Maximum distance for Bluetooth communication is up to 100 meters for a class 1 Bluetooth set. The devices can send a maximum of 3Mb/s. Aside from the fact that the implementation can be highly expensive, we had to overlook this method due to the inconvenience of having to keep the Bluetooth sensor turned on, and thus consuming more power on the part of the Smartphone. This could not match our search for efficiency.

WiFi: WiFi access point fingerprinting is a third approach. It is desirable because it does not necessitate the installation of additional transmitters; it makes use of existing WiFi access points. Though there is no hardware installation requirement, implementing a WiFi fingerprinting based system requires the user to characterize their indoor environment by taking myriad measurements throughout the structure.

Wireless Fidelity (WiFi) is the common nickname for the IEEE 802.11 standard. Wireless connectivity is more prevalent than ever in our everyday lives. Each wireless router broadcasts a signal that is received by devices in the area. Wireless devices have the capability to measure the strength of this signal. This strength is converted to a number, known as received signal strength indicator (RSSI). A user's device can detect the RSSI and MAC address of multiple routers at one time.

RSSI is a dimensionless metric that is used by systems to compare the strength of signals from multiple access points. There is no standard conversion between RSSI and the actual received signal strength (RSS); many manufacturers have their own conversion schemes. Important characteristics of RSSI to RSS conversions include: The maximum and minimum RSSI values (dimensionless integers), the maximum and minimum RSS values that can be represented (dBm), and the resolution of the conversion (value in dBm represented by one RSSI unit). Table 2-1 includes these quantities for common manufacturers.

Manufacturer	RSSI Min	RSSI Max	RSS Min	RSS Max	Resolution
Atheros	0	60	-95dBm	-35dBm	1dBm
Symbol	0	31	-100dBm	-50dBm	10dBm
Cisco	0	100	-113dBm	-10dBm	1dBm

Table A.1 Common RSSI to RSS conversions

The method of conversion is different for each of the manufactures included in Table 2-1. To map an Atheros RSSI value to the associated RSS range, a subtraction of 95 from the RSSI value must be carried out. The Symbol conversion maps ranges of RSSI values to specific RSS values. For example, Symbol RSSI values between 21 and 26 all map to -60dBm. The Cisco conversion is carried out using a table that maps each RSSI value to a specific RSS value. For example, the Cisco RSSI value 35 maps to -77dBm. The Atheros and Cisco WiFi adapters are desirable in applications where accuracy of RSS measurements is important due to the higher resolution of the conversions used by these manufacturers. WiFi devices such as laptops and smartphones typically perform this conversion automatically in order to provide signal strength information to applications running on the device. An advantage of WiFi is that wireless networks are universal. They exist in population-dense areas and are continuously spreading outward. This causes WiFi based systems to have a lower cost of implementation.

ANNEX B INDOOR POSITIONING API

Demo Map Generating using Ericsson Labs' Map Studio

Before generating our map with the Map Editor offered by Ericsson Labs as a part of their Indoor Maps and Positioning API, we had to draw a demo map of our virtual shopping mall in Photoshop, and use it as a background image in Map Editor as to accelerate the map generating process.

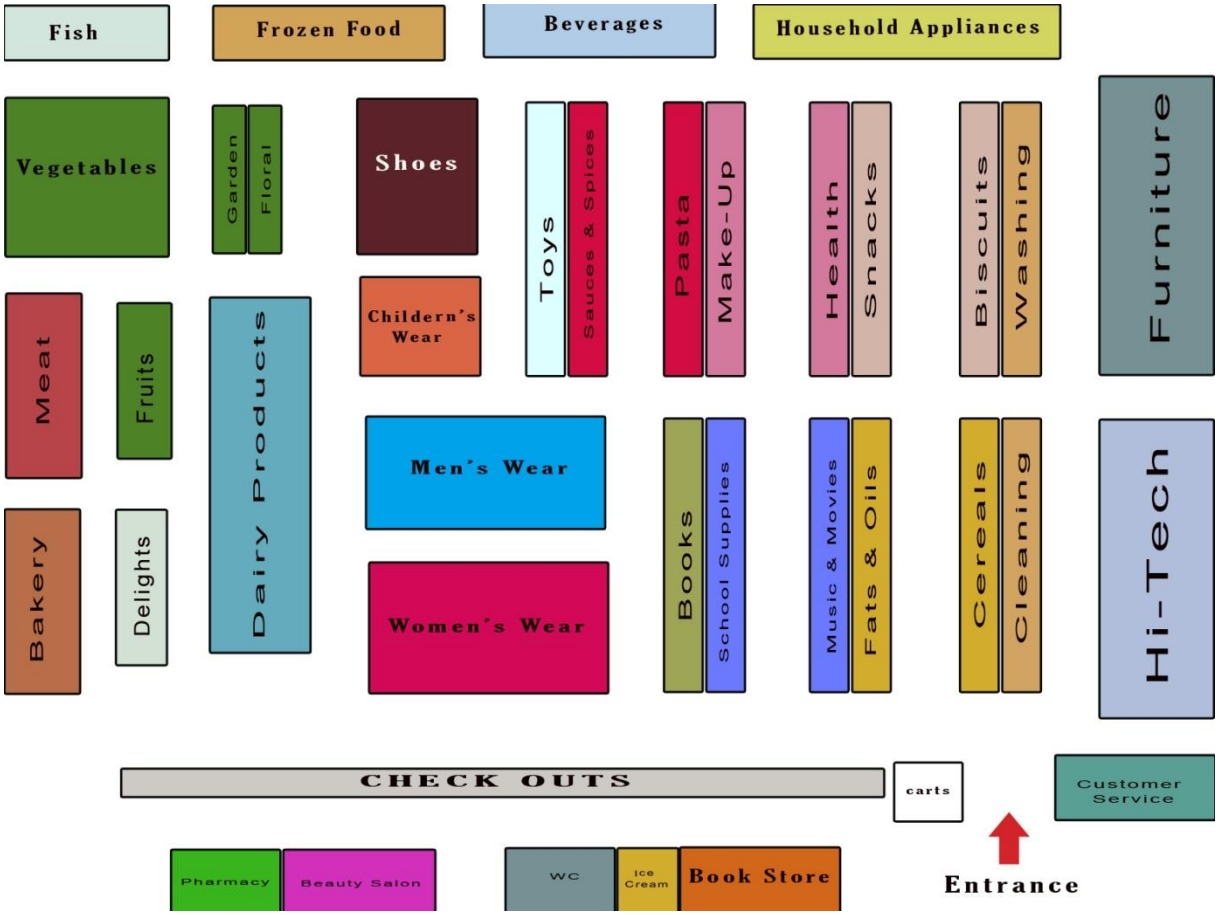


Figure B.1 our customized virtual shopping mall map designed with Photoshop

Before putting geo-locating reference points on the 4 corners of our building, we had to draw the outline and tag the highlighted "way" to "building". We then proceeded to delimiting the perimeter of each area of interest, in our case, the "aisles" and put a significant tag for each area ("vegetables", "pasta", "hi-tech"). We then tagged some nodes in the map as Points of interest (POI) (restroom, entrance). Finally we drew ways on the map and tagged them to "route", as to indicate that theses ways will be considered later by the positioning algorithm. The importance of the tags lies in the fact that they can be considered as map items and will facilitate the positioning later.

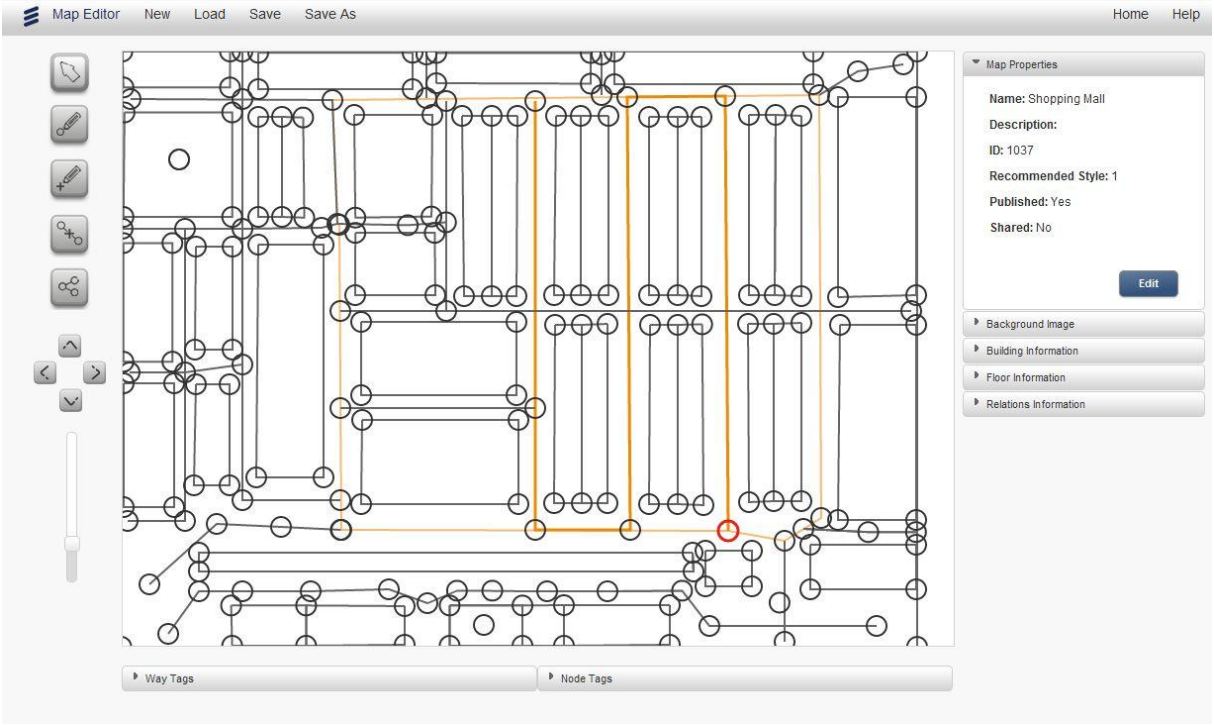


Figure B.2 Our demo map after rendering with the Map Studio

In the figure above we can clearly see the different nodes that make the corners of each area. The orange "ways" have been tagged to "routes".

Indoor Positioning API for Android

The Indoor Positioning API uses Wi-Fi Access Points (APs) to calculate a user's current location in a building. Before you can position yourself, the building's Wi-Fi APs need to be mapped and uploaded to the positioning server. In order to map, you have to tell the positioning server the geographic coordinates of each AP. This is done once you install the Map viewer and Wi-Fi Mapping client on your phone.

The accuracy of a location returned by the Wi-Fi Positioning system is directly dependent on the density, placement and signal power of the APs. The signal strength differs on different AP hardware. The strength is also sensitive to obstacles, such as the human body and walls. The biggest advantage with the use of Wi-Fi as positioning technology is that the APs in most cases already exist. If you want better accuracy or want to provide locations where APs don't already exist, consider installing new ones. They need no Internet connection; they just need to distribute their signal.



Figure B.3 Wi-Fi positioning using triangulation technique

Figure: A location can only be retrieved inside the polygon the Wi-Fi APs creates. For example, if you don't have an AP at the end of a hallway, you'll never receive a location there. The location will be at the AP closest to the end of the hallway.

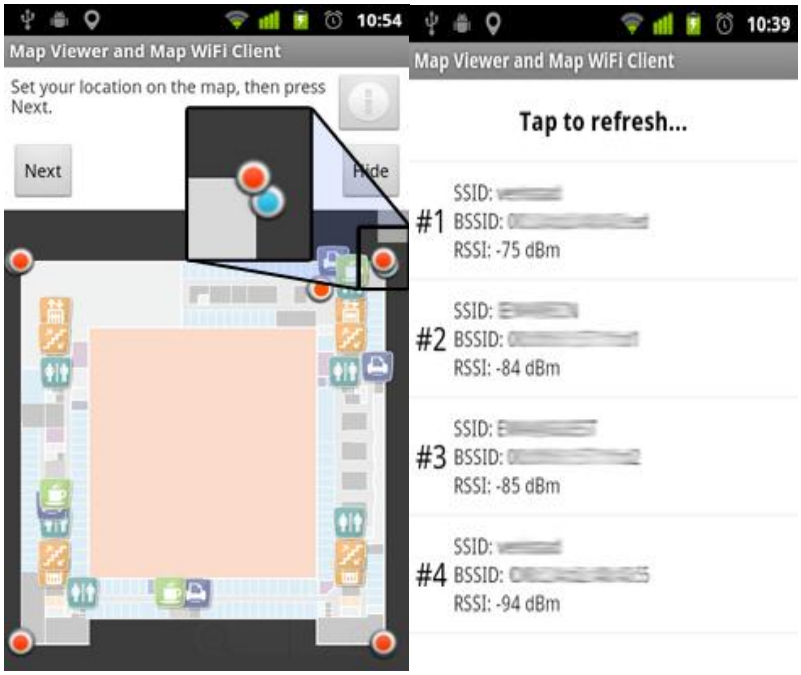


Figure B.4 Screenshots of the Map Viewer and Map Wifi Client

These screenshots show the Reference points with back and forth conversion shown on map followed by a list of reachable access point, sorted by signal strength. The strongest signal is at the top.

ANNEX C REPRESENTATIONAL STATE TRANSFER(REST)

Representational state transfer (REST) is a style of software architecture for distributed systems such as the World Wide Web. REST has emerged over the past few years as a predominant Web service design model. REST has increasingly displaced other design models such as SOAP and WSDL due to its simpler style.

REST attempts to describe architectures that use HTTP or similar protocols by constraining the interface to a set of well-known, standard operations (like GET, POST, PUT, DELETE for HTTP). Here, the focus is on interacting with stateless resources, rather than messages or operations. Clean URLs are tightly associated with the REST concept.
An architecture based on REST can use WSDL to describe SOAP messaging over HTTP, can be implemented as an abstraction purely on top of SOAP (e.g., WS-Transfer), or can be created without using SOAP at all.

REST-style architectures consist of clients and servers. Clients initiate requests to servers; servers process requests and return appropriate responses. Requests and responses are built around the transfer of representations of resources. A resource can be essentially any coherent and meaningful concept that may be addressed. A representation of a resource is typically a document that captures the current or intended state of a resource.

The client begins sending requests when it is ready to make the transition to a new state. While one or more requests are outstanding, the client is considered to be in transition. The representation of each application state contains links that may be used next time the client chooses to initiate a new state transition.